Machine Learning Handbook — Interview Edition

How to use this edition

This is the night-before version of the 20-chapter *Machine Learning Handbook*: every topic, only the facts, formulas and answers you need to say out loud. Nothing here is new — each line is condensed from the full chapter, which has the worked code, plots and longer explanations.

- **One section per chapter**, always in the same order: **Must-know** (one-line definitions) → **Key facts & numbers** (formula and comparison tables) → **Code to remember** (tiny snippets interviewers ask you to write) → **In the interview** (likely questions with 1–3 sentence answers) → **Gotchas** (mistake → correct idea) → **TL;DR**.
- **Formula tables live where the topic lives:** math and gradient descent (Ch 2), regularisation (Ch 5), metrics and bias–variance (Ch 6), impurity (Ch 7), optimizers and initialisation (Ch 10), convolution shapes (Ch 11), attention (Ch 12), RL updates (Ch 14), drift/PSI (Ch 16), fairness and privacy (Ch 19). **All loss functions** and the **"which algorithm" table** are collected in Ch 20.
- **ML system design:** Ch 15 has the seven-step framework and the **napkin-math table**; Ch 17–18 give each of the thirteen case studies as task → features → model → metric → hardest issue.
- **Revision plan:** (1) read the Top 25 below and answer each before reading the answer; (2) skim every chapter's Must-know and TL;DR; (3) study Ch 6 and Ch 15–18 closely if you have a design round; (4) finish with the Last-minute checklist.

Top 25 questions across the book

#	Question	One-line answer	Ch
1	AI vs ML vs deep learning?	AI = intelligent behaviour (incl. rules); ML learns it from data; DL = many-layer nets that learn features	1
2	What is data leakage?	Information unavailable at prediction time reaching training/eval; prevent with point-in-time features, split-first pipelines, time/group splits	4
3	Explain the bias–variance trade-off.	Error = bias² + variance + noise; simple models underfit, flexible ones overfit; U-shaped in complexity	6
4	How do you detect and fix overfitting?	Train ≫ validation score; more data, regularisation, simpler model, early stopping, bagging, augmentation	6
5	Explain gradient descent and the learning rate.	$w \leftarrow w - \eta \nabla L$; too small is slow, too large diverges; mini-batches; scale features	2
6	Bayes' theorem with an example?	Posterior ∝ likelihood × prior; 90%-sensitive, 5%-FPR test at 1% prevalence → ~15% P(sick | +)	2
7	Why cross-entropy for classification?	Maximum-likelihood loss, convex for LR, punishes confident errors, gradient p − y	2, 10
8	Ridge vs Lasso?	L2 shrinks and shares weight; L1 zeroes weights (selection); more penalty → more bias, less variance	5
9	Linear vs logistic regression?	MSE on a number vs sigmoid + log-loss on a probability; boundary w·x + b = 0; e^w = odds ratio	5
10	Precision vs recall — which to optimise?	By error cost; threshold p > C_FP/(C_FP + C_FN) with calibrated scores; two thresholds for fraud	6
11	ROC-AUC vs PR-AUC?	ROC ignores base rate; with rare positives PR-AUC (no-skill = base rate) exposes false alarms	6
12	What is calibration and when does it matter?	Probabilities match frequencies; needed when scores feed arithmetic (auctions, expected cost); Platt/isotonic	6, 18
13	k-fold CV and its variants?	k rounds, mean ± std; stratified, group, time-series with gap; nested CV for tuned estimates	6
14	Bagging vs boosting?	Parallel deep trees averaged (variance) vs sequential shallow trees fixing errors (bias)	7
15	Why can feature importance mislead?	MDI is on training data, favours high cardinality, splits credit among correlated features → permutation/SHAP	7
16	Explain SVMs and the kernel trick.	Maximise margin $2/\|w\|$ with C; dual needs only dot products, so K(x, z) gives non-linear boundaries	8
17	How does k-means work; how to choose k?	Assign/update to local optimum; k-means++ + restarts; silhouette/elbow; assumes round clusters	9
18	How does backpropagation work?	Chain rule from the loss backwards reusing activations; ~one extra forward pass for all gradients	10
19	Vanishing/exploding gradients — fixes?	ReLU, He/Xavier init, normalisation, residuals, gating (LSTM/GRU), clipping	10, 12
20	Explain attention and the transformer.	$\text{softmax}(QK^T/\sqrt{d_k})V$ weighted lookup; multi-head self-attention + FFN with residual + LayerNorm; parallel, $O(n^2)$	12
21	Candidates from 100 M items in ms?	Two-tower; items in an ANN index (HNSW/IVF); user tower per request; logQ; versioned with the index; then a ranker	13

#	Question	One-line answer	Ch
22	Q-learning vs SARSA?	Off-policy max target (optimal policy) vs on-policy next-action target (safer while exploring)	14
23	Walk me through designing an ML system.	Frame → success metrics → data/labels → baseline → serving → keep it working → failures and cost	15
24	How do you monitor and when do you retrain?	Health → input drift (PSI/KS/nulls) → prediction drift → outcomes; investigate; gated retraining	16
25	Your model is 94% accurate — is it fair?	Slice by group: selection rate, TPR, FPR, PPV, calibration, proxies; criteria conflict; choose by harm	19

1. What Machine Learning Is

The vocabulary and the loop: what ML is, the learning setups, the workflow and the ways it fails.

Must-know

- **AI ⊃ ML ⊃ DL** — AI is the goal (includes rules and search); ML learns behaviour from data; deep learning is ML with many-layer neural nets that learn their own features. **Generative AI** creates content; **agentic AI** plans and acts with tools in a loop.
- **Rules vs learning** — programming: people write rules; ML: rules come from data. Use ML when rules are too complex or change too often *and* you have good examples. Try a simple rule as the baseline first.
- **Training / inference / feedback loop** — training adjusts parameters to reduce loss; inference applies the model to new data; feedback loops turn real outcomes into new training data.
- **Supervision types** — supervised (labels), unsupervised (none: clusters, dimensions, anomalies), semi-supervised (few labels + many unlabelled), **self-supervised** (labels made from the data, e.g. next token — LLM pre-training), reinforcement (rewards from actions).
- **Classification vs regression** — category vs number; the label type decides loss, metrics and algorithms.
- **Parameters vs hyperparameters** — learned in training (w, b) vs chosen before it (degree, C, learning rate, number of trees) and tuned on validation data.
- **Train / validation / test** — train fits parameters, validation picks models and hyperparameters, **test is used once** so its score stays honest.
- **Batch vs online learning** — retrain on everything periodically vs update continuously from small chunks (`partial_fit`); online adapts faster but needs careful monitoring.
- **Instance-based vs model-based** — k-NN keeps and compares stored examples; model-based fits parameters and discards the data.
- **Workflow** — frame → data → features → model → evaluate → deploy → monitor, in a loop.
- **Training–serving skew** — features computed differently in production than in training, so the model sees unfamiliar inputs.
- **Main failures** — too little / noisy / non-representative (sampling bias) / irrelevant data; overfitting; underfitting; **data leakage** (too-good-to-be-true scores).

Key facts & numbers

Train error	Validation error	Diagnosis	First fixes
High	High (close)	Underfitting (high bias)	More capacity, better features, less regularisation, train longer
Low	Much higher	Overfitting (high variance)	More data, more regularisation, simpler model, early stopping, bagging
Low	Low (close)	Good fit	Test once, ship
Suspiciously perfect	Suspiciously perfect	Probably **leakage**	Check timestamps, splits, duplicates, top importances

Loss on 3 houses (true 300, 250, 400; predicted 310, 240, 380, in $1000s)	Value
MSE = (100 + 100 + 400)/3	200
RMSE = √200	≈ 14.1 ("off by about $14k")
MAE = (10 + 10 + 20)/3	≈ 13.3

Deep learning took off around 2012 because three things arrived together: large labelled datasets (ImageNet), GPUs, and training tricks (ReLU, dropout, better initialisation).

Code to remember

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
print(cross_val_score(pipe, X_tr, y_tr, cv=5).mean())   # compare to a DummyClassifier baseline
pipe.fit(X_tr, y_tr); print(pipe.score(X_te, y_te))      # test once
joblib.dump(pipe, "model.joblib")                          # save scaler + model together
```

In the interview

Q: AI vs ML vs deep learning? **A:** AI is intelligent behaviour in general (rules count); ML learns it from data; DL is ML with deep neural nets that learn features from raw pixels/text.

Q: When classic ML over deep learning? **A:** Small/medium tabular data, a need for interpretability, tight latency or budget.

Q: Is LLM pre-training unsupervised? **A:** Better called self-supervised — there is a target (the next token), generated from the data. Then supervised fine-tuning, then RL from human/AI feedback.

Q: Is anomaly detection supervised? **A:** Usually unsupervised or semi-supervised (trained mostly on normal data), because labelled anomalies are rare.

Q: Walk me through an ML project. **A:** Frame the decision, metric and baseline; hold out a test set early; build a preprocessing + model pipeline; compare families with CV; tune on validation; test once; deploy the pipeline as a unit; monitor drift and retrain.

Q: How do you detect overfitting? **A:** Low training error, much higher validation error; learning curves.

Q: More data or a better model? **A:** Overfitting → more data helps; underfitting → more of the same data barely helps, you need capacity or features.

Gotchas

- Saying "AI" for a specific technique → name it ("a gradient-boosted classifier").
- Newer = better → on tabular business data tree ensembles often beat deep nets.
- Treating clusters as true categories without checking.
- Encoding red/green/blue as 1/2/3 and doing regression → invents an order.
- Accuracy on imbalanced data → "always legitimate" is 99% accurate and useless.
- Reporting the tuned CV score as final → slightly optimistic; the test set is the honest number.
- Saving only the model, not the scaler → unscaled inputs in production. Save the whole pipeline.

TL;DR

Features in, label out; training minimises a loss, validation picks hyperparameters, test is used once. Baseline first, pipeline everything, split correctly, monitor always.

2. The Math You Need for Machine Learning

Just enough linear algebra, calculus, probability, statistics and information theory to read every model.

Must-know

- **Vector / dot product** — a data point is a vector; $a \cdot b = \Sigma a_i b_i$ is how a linear model combines features; cosine similarity measures direction.
- **Matrix** — a dataset (rows = examples); predictions for all rows = **$Xw + b$**; a neural-net layer is a matrix multiply + nonlinearity. **$AB \neq BA$**; inverse exists only if square and det ≠ 0.
- **Eigenvector / SVD** — directions a matrix only stretches ($Av = \lambda v$); SVD $M = U\Sigma V^T$ = rotate–stretch–rotate; top-k singular values = best rank-k approximation. Powers PCA, compression, recommenders.
- **Gradient** — vector of partial derivatives, points **uphill**; step along minus it.
- **Chain rule** — multiply slopes through nested functions; the engine of backpropagation.
- **Gradient descent** — $w \leftarrow w - \eta \nabla L$; mini-batch SGD is the practical default; epoch = one pass over the data.
- **Distributions** — Bernoulli (one yes/no), binomial (count of yeses), normal (bell); expectation = long-run average, variance = average squared spread.
- **Bayes** — posterior ∝ likelihood × prior; mind the **base rate**.
- **Robust statistics** — median/IQR resist outliers; mean/std don't. Standardise before distance- or gradient-based models.
- **Correlation ≠ causation** — confounders, reverse causation, chance; causes need randomised experiments.
- **CLT** — sample means are ~normal around the truth with spread **$\sigma/\sqrt{n}$**; representative beats big.
- **p-value** — P(data this extreme | H_0). Not P(H_0 true). Significant ≠ important.
- **Entropy / cross-entropy / KL** — uncertainty / surprise when predicting with q / the gap (≥ 0, not symmetric).

Key facts & numbers

Idea	Formula	Worked value
Dot product	$a \cdot b = \Sigma a_i b_i = \|a\|\|b\|\cos\theta$	$[1,2,3]\cdot[4,5,6] = 32$
Cosine similarity	$a \cdot b / (\|a\|\|b\|)$	≈ 0.975 for the pair above
2×2 inverse	$\det = ad - bc$; $A^{-1} = (1/\det)[[d, -b], [-c, a]]$	$[[1,2],[3,4]]$: $\det -2$
Normal equation	$\theta = (X^TX)^{-1}X^Ty$ (use `lstsq`); Ridge $(X^TX + \alpha I)^{-1}X^Ty$	
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$; $\sigma' = \sigma(1 - \sigma)$	$\sigma(0) = 0.5$, $\sigma'(0) = 0.25$ (max)
Gradient descent	$w \leftarrow w - \eta\nabla L$	$f = (w-3)^2$, η 0.1: 0 → 0.6 → 1.08
MSE gradient	$(2/n)X^T(Xw - y)$	
Log-loss gradient	$(1/n)X^T(\sigma(Xw) - y)$ — features × (prediction − truth)	
Numerical check	$f'(x) \approx (f(x+h) - f(x-h))/2h$	
Convergence (MSE)	$\eta < 2/\lambda_{max}$ of the Hessian	
Binomial	mean np, var np(1−p)	P(3 of 10, p 0.3) ≈ 0.2668
Bayes	$P(A\|B) = P(B\|A)P(A)/P(B)$	1% prevalence, 90% sensitivity, 5% FPR → P(sick|+) ≈ 0.154; 2nd positive → 0.766
Standard error	$\sigma/\sqrt{n}$; 4× data halves it	σ 20, n 100 → 2
CI	estimate ± 1.96·SE (95%)	bootstrap: 2.5th–97.5th percentiles
Bonferroni	α/m	20 tests at 0.05 → expect 1 false hit
Entropy	$H(p) = -\Sigma p \log p$	fair coin 1 bit; 90/10 coin 0.469
Cross-entropy / KL	$H(p,q) = -\Sigma p \log q \geq H(p)$; $KL = H(p,q) - H(p)$	
Log loss	$-(1/n)\Sigma[y \ln p + (1-y) \ln(1-p)]$	$[1,0,1,1]$ vs $[0.9,0.2,0.6,0.99]$ → 0.2123 nats

NumPy std uses `ddof=0` (÷n), pandas `ddof=1` (÷(n−1)). Pearson r = 0 does not mean independent ($y = x^2$ can give r = 0); Spearman uses ranks.

Code to remember

```
w = 0.0
for _ in range(100):
    grad = 2 * (w - 3)        # d/dw of (w-3)^2
    w -= 0.1 * grad           # step opposite the gradient
# vectorised linear regression: grad = 2/n * X.T @ (X @ w - y)
```

In the interview

Q: Why do matrices matter in ML? **A:** A dataset is a matrix and predictions are one product Xw + b; layers are matrix multiplies, which is why GPUs dominate deep learning.

Q: Explain gradient descent. **A:** Move parameters a small step opposite the gradient, $w \leftarrow w - \eta\nabla L$; too small η is slow, too large diverges; use mini-batches and scale features.

Q: Does GD always find the global minimum? **A:** Only guaranteed for convex problems with a suitable learning rate.

Q: Why mini-batches? **A:** Cheaper than full batch, less noisy than single examples, fits GPU parallelism.

Q: Bayes with an example. **A:** 90%-sensitive, 5%-FPR test, 1% prevalence → only ~15% P(sick | positive); that is exactly the test's precision — why rare-event classifiers have low precision.

Q: What is a p-value? **A:** Probability of data at least this extreme if H_0 were true; reject if below α. Report effect sizes with CIs and correct for multiple tests.

Q: Comparing two models' accuracy? **A:** Same test set, paired test (McNemar) or bootstrap the difference.

Q: Why cross-entropy for classification? **A:** It is the maximum-likelihood loss, punishes confident errors hardest, gradient p − y at the logits.

Gotchas

- `a * b` is element-wise; `a @ b` is the dot/matrix product.
- Distances on unscaled features → the large feature dominates.
- Inverting near-singular X^TX (multicollinearity) → use `solve` / `lstsq` or regularise.
- Chain rule: forgetting the inner derivative $((3x+1)^2)' = 2(3x+1)\cdot 3$.
- Loss NaN or growing → divide η by 10.
- P(A|B) ≠ P(B|A); ignoring the prior.
- A PDF value is not a probability; P(X = exactly 170) = 0.
- Peeking at an A/B test until p < 0.05 inflates false positives.

TL;DR

$Xw + b$, $w \leftarrow w - \eta\nabla L$, chain rule, posterior ∝ likelihood × prior, $\sigma/\sqrt{n}$, and cross-entropy — that's the math the rest of the book runs on.

3. The Python ML Stack

NumPy, pandas, matplotlib, the scikit-learn estimator API, Jupyter habits, and when to reach for PyTorch.

Must-know

- **Environment** — one virtual environment per project; `python -m pip install ...` so pip matches the interpreter; freeze versions.
- **ndarray** — homogeneous, contiguous, typed memory; operations run in compiled code, so **vectorised** code is orders of magnitude faster than Python loops.
- **Broadcasting** — align shapes from the right; each dimension equal or 1. (3,1)+(1,4) → (3,4); (3,2)+(3,) → error.
- **Views vs copies** — basic slices are views; boolean and fancy indexing return copies.
- **axis=0** — operate down the rows → one result per column.
- **pandas** — labelled, column-typed tables on NumPy. `loc` = labels (end inclusive), `iloc` = positions (end exclusive). `agg` = one row per group; `transform` = same length as input.
- **pandas 3 copy-on-write** — assign (`df["c"] = df["c"].fillna(0)`), don't chain in-place calls.
- **Estimator API** — hyperparameters in the constructor; `fit` returns self; learned attributes end in `_`; transformers add `transform`, predictors add `predict`. `score` = accuracy (classifiers), R^2 (regressors).
- **Pipeline + ColumnTransformer** — leak-free (refit per CV fold), tunable via `step__param`, one artifact to deploy. `ColumnTransformer` default `remainder="drop"`.
- **Frameworks** — scikit-learn (+ XGBoost/LightGBM/CatBoost) for tables; PyTorch or TensorFlow/Keras for images, text, audio, GPUs and custom nets.
- **Jupyter** — seed first, run top to bottom, Restart & Run All before sharing, move stable code to `.py` modules.

Key facts & numbers

Default (scikit-learn 1.9)	Value
`LogisticRegression`	C = 1.0 (L2), solver `lbfgs`
`RandomForestClassifier`	`max_features="sqrt"` (regressor 1.0)
`HistGradientBoosting*`	learning_rate 0.1, `max_leaf_nodes` 31, `min_samples_leaf` 20
`SVC`	RBF kernel, C = 1, `gamma="scale"`
`KNeighbors*`	k = 5, Minkowski p = 2
`KMeans`	`init="k-means++"`

Code to remember

```
pre = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median", add_indicator=True), StandardScaler()), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])
pipe = Pipeline([("pre", pre), ("model", LogisticRegression(max_iter=1000))])
search = GridSearchCV(pipe, {"model__C": [0.01, 0.1, 1, 10]}, cv=StratifiedKFold(5), scoring="average_precision")
search.fit(X_train, y_train)          # preprocessing refit inside every fold
print(search.best_params_, search.score(X_test, y_test))   # test once
```

In the interview

Q: Why is NumPy fast? **A:** Homogeneous contiguous arrays processed in compiled code; vectorisation + broadcasting avoid the interpreter loop.

Q: View or copy? **A:** Basic slices are views (writes change the original); boolean/fancy indexing copies.

Q: Your first steps on a new DataFrame? **A:** `head`, `dtypes`, `info`, `describe`, `isnull().sum()`, fix types, handle missing values deliberately, then `groupby` and plots.

Q: Avoid a merge blowing up rows? **A:** Check key uniqueness and pass `validate="one_to_one"` / `"many_to_one"`.

Q: `fit_transform` vs `transform`? **A:** `fit_transform` learns statistics and applies them (training only); `transform` applies learned statistics to val/test/production.

Q: Tune a hyperparameter inside a pipeline? **A:** `GridSearchCV` with keys like `model__C` or `prep__num__impute__strategy`.

Q: Why Pipeline? **A:** CV refits preprocessing per fold (no leakage), one object to tune, save and deploy.

Gotchas

- Shape (n,) vs (n, 1) — scikit-learn wants 2-D `X`: `x.reshape(-1, 1)`.
- (n,) − (n, 1) silently broadcasts to (n, n).
- `and` / `or` in pandas filters → use `&`, `|`, `~` with parentheses.
- Row-wise `apply` on big data → vectorise.
- `fit` on the test set; preprocessing the full dataset before CV → leakage.
- Missing `handle_unknown="ignore"` → crash on a new category in production.
- `LabelEncoder` is for targets, not features.

TL;DR

Typed arrays + vectorise; check dtypes and nulls first; constructor = hyperparameters, `fit` learns `attr_`; wrap everything in a Pipeline.

4. Data, EDA and Feature Engineering

Where labels come from, how to look at and clean data, encode/scale/create features, split correctly, and avoid leakage and imbalance traps.

Must-know

- **Label sources** — outcomes (late), behaviour/implicit (fast but biased by exposure and position), human annotation (costly), weak supervision (labelling functions + label model; noisy), LLMs (scalable, correlated errors — audit with human slices).
- **Cohen's κ** — chance-corrected agreement (p_o − p_e)/(1 − p_e); report it, not raw percent agreement. Version the labelling guideline.
- **EDA** — shape & types → missingness (related to target?) → distributions (skew, sentinels like 999) → target balance → relationships (Spearman for skewed) → duplicates, impossible values, leaky IDs. Do detailed EDA on the **training split**.
- **Missingness** — MCAR / MAR / MNAR; default median (numbers) or `"missing"` (categories) **+ missing indicator**; KNN/iterative imputation when other columns predict it; GBDTs handle NaN natively.
- **Encoding** — one-hot (small nominal), ordinal (ordered; fine for trees), **target encoding** (high cardinality; cross-fitted + smoothed or it leaks), hashing (unbounded IDs; collisions), embeddings (deep models).
- **Scaling** — needed for anything using distances, gradients or penalties (k-NN, k-means, SVM, regularised linear, neural nets, PCA); not for trees. Scaling never fixes skew → log.
- **Feature groups (production)** — entity (batch), behavioural aggregates (streaming), context (request time), cross (pairs). Every feature must exist at prediction time (**point-in-time correct**).
- **Splits** — random (i.i.d.), stratified (classification), **time-based** (predicting the future), **group** (new users/patients). Millions of rows → 98/1/1 fine; a few thousand → CV.
- **Data leakage** — information unavailable at prediction time reaching training/evaluation. Split first, fit everything inside a `Pipeline`.
- **Imbalance** — breaks accuracy, not learning; change the metric, move the threshold, weight or resample the **training** data, recalibrate.

Key facts & numbers

Scaler	Formula	Use when
Standard	(x − mean)/std	Default for linear, SVM, NN, PCA, k-NN, k-means
Min-max	(x − min)/(max − min)	Bounded inputs (pixels ÷ 255), no big outliers
Robust	(x − median)/IQR	Outliers you keep
Log / power	log1p(x), `PowerTransformer`	Heavy right tails

Technique	Formula / example
κ	agreement 0.8, chance 0.5 → 0.6; 90% agreement when both say "not spam" 90% → p_e 0.82, κ ≈ 0.44
Outlier fences	Q1 − 1.5·IQR, Q3 + 1.5·IQR; or |z| > 3
Cyclical time	hour h → (sin 2πh/24, cos 2πh/24) so 23 and 0 are neighbours
TF-IDF (sklearn)	idf(t) = ln((1+n)/(1+df)) + 1, rows L2-normalised
Polynomial blow-up	`PolynomialFeatures(2)` on 10 features → 65 columns
Downsampling correction	**p_true = p·w / (p·w + 1 − p)**; w = 0.1, p = 0.5 → 0.091
SMOTE	new point a + u·(b − a) between minority neighbours; training data only

Imbalance technique	Effect on probabilities
Threshold moving	None — often all you need
Class weights	Inflated → recalibrate
Negative downsampling	Inflated → correct with formula above
Oversampling / SMOTE	Inflated; after splitting only

Leakage checklist: (1) known at prediction time? (2) split first, pipeline everything; (3) time/group splits if deployment needs them; (4) dedupe across splits; (5) aggregate windows that include the label moment; (6) label-adjacent fields (refund flag, resolution code); (7) distrust a single dominant feature or a score far above published baselines.

Code to remember

```
from sklearn.preprocessing import TargetEncoder     # cross-fits internally in fit_transform
pre = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median", add_indicator=True), StandardScaler()), num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), low_card),
    ("hi",  TargetEncoder(), high_card),
])
pipe = make_pipeline(pre, LogisticRegression(class_weight="balanced", max_iter=1000))
```

In the interview

Q: How will you get labels? **A:** List sources (outcomes, behaviour, human review, rules, LLMs) with cost, delay and bias; check quality with κ and gold questions; train only on matured labels joined to features as they were at prediction time.

Q: Why is a click a weak label? **A:** It measures the old system's exposure and position as much as preference; a non-click is not a dislike.

Q: Handling missing values? **A:** Ask why they're missing; median + indicator as a default; KNN/iterative if other columns predict it; fit inside the pipeline; judge by masking known values and comparing CV scores.

Q: Hashing vs target encoding? **A:** Hashing: no labels, no state, handles new values, but collides; target encoding: compact and informative but must be out-of-fold and can go stale.

Q: Why is ordinal encoding OK for trees? **A:** Trees split on thresholds and can isolate any level; the false order costs a few extra splits.

Q: Does logistic regression need scaling? **A:** Not for correctness, but yes for convergence speed and fair regularisation.

Q: Offline fraud AUC is 0.99 — what first? **A:** Feature timestamps vs label time, split method, duplicates, top importances.

Q: Downsample or reweight? **A:** Downsample when data is huge and compute matters; reweight when modest. Both need recalibration.

Gotchas

- Imputation/scaler/bin/TF-IDF statistics fitted on all data → leakage; fit on train only.
- Deleting real outliers (fraud *is* the outlier); age 999 is an error → NaN, not an outlier.
- Mode-filling categoricals loses the "was missing" signal.
- Month/hour as a plain number → December far from January.
- Random splits on time data → training on the future.
- Oversampling before splitting → copies in train and test; never resample the test set.
- Reading a downsampled model's scores as real probabilities (ad auctions, expected cost).
- `FunctionTransformer` with data-dependent logic recomputes per batch → use a fitted transformer.

TL;DR

If the model couldn't know it at prediction time, it mustn't see it in training. Split first (time/group when needed), pipeline every transform, pick encoders by cardinality, and fix imbalance with metrics and thresholds before resampling.

5. Linear and Logistic Regression

The first real models: MSE, the normal equation vs gradient descent, assumptions, regularisation, logistic and softmax regression, and reading coefficients.

Must-know

- **Linear regression** — $\hat{y} = w \cdot x + b$, minimise MSE (a convex bowl, one minimum).
- **Normal equation** — $\theta = (X^TX)^{-1}X^Ty$: exact, no tuning, $O(d^3)$ — use `lstsq` / `solve`, never an explicit inverse. Fails when X^TX is singular (collinear features, $d > n$) → pseudo-inverse or Ridge.
- **Gradient descent** — $\theta \leftarrow \theta - \eta(2/m)X^T(X\theta - y)$: scales to huge data and any differentiable loss; needs a learning rate and **scaled features** (else an elongated bowl → zig-zag).
- **GD variants** — batch (smooth, costly), stochastic (noisy, needs decaying η), **mini-batch** (default, fits GPUs).
- **Assumptions** — linearity, independent errors, constant variance, roughly normal errors, no strong multicollinearity. Residuals vs fitted: **U** = missing curvature, **funnel** = heteroscedasticity.
- **Polynomial regression** — linear model on powers/products; linear in the weights; degree = complexity knob; wild at the edges.
- **Ridge (L2)** — shrinks all weights, shares weight among correlated features. **Lasso (L1)** — exact zeros = feature selection (constant slope; diamond corners on the axes). **ElasticNet** — both. More regularisation → more bias, less variance.
- **Logistic regression** — $p = \sigma(w \cdot x + b)$, log-loss (convex), gradient $X^T(p - y)/m$, boundary **$w \cdot x + b = 0$**, coefficient = change in log-odds, **e^w = odds ratio**.
- **Softmax regression** — K scores → $e^{z_k}/\Sigma e^{z_j}$ + cross-entropy; coherent probabilities for exclusive classes. OvR = K binary models (works for multi-label, any binary model).
- **Coefficients** — effect per unit holding others fixed; standardise to compare; unstable under multicollinearity; associations, not causes.

Key facts & numbers

Formula	Meaning
$MSE = (1/n)\Sigma(\hat{y} - y)^2$	Loss; optimal constant = mean; MLE under Gaussian noise
Ridge loss	MSE + $\alpha\Sigma w^2$; closed form $(X^TX + \alpha I)^{-1}X^Ty$
Lasso loss	MSE + $\alpha\Sigma\|w\|$
$\sigma(z) = 1/(1 + e^{-z})$	Sigmoid; log-odds = $w \cdot x + b$
Log loss	$-[y \ln p + (1-y) \ln(1-p)]$
C in `LogisticRegression`	**inverse** of strength; default C = 1 (L2); `C=np.inf` = no penalty (sklearn ≥ 1.8)
Softmax stability	subtract max z first (e^1000 overflows)
Softmax with 2 classes	= logistic regression on the score difference
GD convergence (MSE)	$\eta < 2/\lambda_{max}$

	Normal equation	Gradient descent
Cost	$O(nd^2 + d^3)$	$O(nd)$ per epoch
Tuning	none	learning rate, iterations
Scaling needed	no	yes
Good for	small d	huge n or d, any differentiable loss

Code to remember

```
X_b = np.c_[np.ones(len(X)), X]                      # add the bias column
theta, *_ = np.linalg.lstsq(X_b, y, rcond=None)      # normal equation, stably
theta = np.zeros(X_b.shape[1])
for _ in range(1000):                                # batch gradient descent
    theta -= eta * 2 / len(y) * X_b.T @ (X_b @ theta - y)
# logistic: grad = X_b.T @ (sigmoid(X_b @ theta) - y) / len(y)
```

In the interview

Q: Normal equation vs gradient descent? **A:** Exact $\theta = (X^TX)^{-1}X^Ty$, $O(d^3)$, no tuning — small d; GD scales and generalises to any differentiable loss but needs η and scaled features.

Q: Is MSE convex for linear regression? **A:** Yes; GD with small enough η reaches the global minimum.

Q: Why does SGD sometimes generalise better? **A:** Its noise acts like a regulariser and can escape sharp minima (empirical, not guaranteed).

Q: Funnel-shaped residuals? **A:** Heteroscedasticity — log-transform y, weighted least squares or robust standard errors.

Q: Ridge vs Lasso? **A:** L2 shrinks smoothly and keeps correlated features; L1 zeroes weights. Two identical features: Ridge splits the weight equally, Lasso picks one. Scale first, tune α by CV.

Q: Why not MSE for classification? **A:** With a sigmoid it is non-convex and gives weak gradients for confidently wrong predictions; log-loss is convex and punishes them hard.

Q: Perfectly separable data? **A:** Unregularised weights diverge as the loss keeps falling; regularisation keeps them finite.

Q: Is logistic regression calibrated? **A:** Often roughly (log-loss is a proper scoring rule), but check a reliability curve.

Q: Interpret a logistic coefficient. **A:** +1 unit of x adds w to the log-odds; odds multiply by e^w, holding others fixed.

Gotchas

- `np.linalg.inv(X.T @ X)` → slower and less stable than `lstsq`.
- Regularising unscaled features; tuning α on the test set (use `RidgeCV` / `LassoCV`).
- `LinearRegression` on a 0/1 target → predictions outside [0, 1].
- Assuming 0.5 is the right threshold on imbalanced data.
- Checking only R^2, never residuals.
- Comparing raw coefficients across units, or reading them causally; regularised coefficients are biased toward 0.

TL;DR

Linear: MSE, solve exactly for small d or GD for big. Logistic: sigmoid + log-loss, boundary $w \cdot x + b = 0$, e^w = odds ratio. L2 shrinks, L1 selects — scale first, tune by CV.

6. Evaluation and Tuning

The most-asked chapter: baselines, metrics, thresholds, calibration, cross-validation, bias–variance, curves, tuning and early stopping.

Must-know

- **Baselines first** — majority class / mean / last value / most popular, plus the production model. Offline metrics shortlist; **online experiments decide**.
- **MAE** — typical error in target units (median-optimal). **RMSE** — squares first, punishes big misses (mean-optimal); RMSE ≫ MAE signals heavy-tailed errors. **R^2** — variance explained vs predicting the mean (not comparable across datasets). Add **p90 error** and **per-cohort** errors.
- **Confusion matrix → precision / recall / F1** — precision = how many alerts are real; recall = how many positives caught; F1 = harmonic mean (stays low if either is low).
- **Threshold** — the model gives scores, you choose the operating point from error costs; fraud/moderation use **two thresholds** (approve / review / block).
- **ROC-AUC** — P(random positive outranks random negative); base-rate insensitive. **PR-AUC** — exposes false alarms; the choice for rare positives (no-skill = base rate).
- **Ranking** — recall@k for retrieval, NDCG@k (graded, position-discounted) or MRR (first good answer) for ranking, MAP for binary relevance.
- **Calibration** — predicted probabilities match observed frequencies; separate from ranking. Reliability diagram + Brier/log-loss; fix with **Platt** (small data) or **isotonic** (≳ 1,000 examples) on held-out data. AUC unchanged by any increasing map.
- **Cross-validation** — k rounds, report **mean ± std**; stratified (classes), group (entities), `TimeSeriesSplit` with a gap (time); preprocessing inside the pipeline; nested CV or a test set for the final number.
- **Bias–variance** — test error = **bias² + variance + noise**; U-shaped in complexity. Regularisation: bias ↑ variance ↓. Bagging cuts variance, boosting cuts bias.
- **Learning curve** (score vs data size) → will more data help? **Validation curve** (score vs one hyperparameter) → sweet spot.
- **Tuning** — grid (1–2 params), random (default for 3+), successive halving/Hyperband (prune early), Bayesian/Optuna (expensive trials); log scales; CV only; test once.
- **Early stopping** — stop after `patience` rounds with no validation improvement, restore best; never on the test set.

Key facts & numbers

Metric	Formula	Worked value
MAE / MSE / RMSE / R^2	$(1/n)\Sigma\lvert e\rvert$ / $(1/n)\Sigma e^2$ / √MSE / $1 - \Sigma e^2/\Sigma(y-\bar{y})^2$	y (30,25,40,35,50), ŷ (28,27,35,35,60): 3.8 / 26.6 / 5.16 / 0.641
Accuracy	(TP+TN)/all	TP 60, FP 20, FN 40, TN 880 → 0.94 ("flag nothing" 0.90)
Precision	TP/(TP+FP)	0.75
Recall (TPR)	TP/(TP+FN)	0.60
F1 / Fβ	$2PR/(P+R)$ / $(1+\beta^2)PR/(\beta^2P+R)$	0.667; F2 favours recall; P 1, R 0.01 → F1 ≈ 0.02
Specificity / FPR	TN/(TN+FP) / FP/(FP+TN)	0.978 / 0.022
ROC-AUC	P(pos > neg)	pos (0.9,0.7,0.4) vs neg (0.8,0.3,0.2) → 7/9 = 0.778
Brier	mean((p − y)²)	always 0.5 → 0.25
Cost threshold	flag if **p > C_FP/(C_FP + C_FN)**	review $5, missed fraud $100 → p > 0.048
MRR	mean 1/rank of first relevant	
NDCG@k	DCG/IDCG, DCG = $\Sigma\ rel_i/\log_2(i+1)$ (or 2^rel − 1 gain)	always state the gain and the ideal
CV averaging	$\rho\sigma^2 + (1-\rho)\sigma^2/B$ for B models	bagging shrinks 2nd term, feature subsets shrink ρ

Situation	Primary metric
Balanced, equal costs	Accuracy + confusion matrix
False positives costly	Precision at a recall floor
Misses costly	Recall at a precision floor
Rare positives (≤ ~1%)	PR-AUC + P/R at the threshold
Probabilities used in arithmetic	Log loss, Brier, ECE per segment
Multiclass, weak minority classes	Macro-F1
Retrieval / ranking	Recall@k / NDCG@k
Launch decision	Online business metric in an A/B test + guardrails

Code to remember

```
cv = StratifiedKFold(5, shuffle=True, random_state=0)       # GroupKFold / TimeSeriesSplit(gap=...) as needed
s = cross_val_score(pipe, X, y, cv=cv, scoring="average_precision")
print(f"{s.mean():.3f} ± {s.std():.3f}")
cal = CalibratedClassifierCV(model, method="isotonic", cv=5).fit(X_tr, y_tr)
search = RandomizedSearchCV(pipe, {"model__C": loguniform(1e-3, 1e2)}, n_iter=30, cv=cv)
```

In the interview

Q: Which metric would you use? **A:** Name the task family, base rate and cost asymmetry; give an operating-point metric (precision at recall X) plus a threshold-free one (PR-AUC or NDCG); calibration if probabilities are consumed; compare to naive and production baselines; confirm online.

Q: Offline metric up, A/B flat — why? **A:** The proxy drifted from the goal (proxy labels, stale test set, training/serving differences); also check power and logging.

Q: PR-AUC or ROC-AUC at 0.2% base rate? **A:** PR-AUC — precision (what reviewers experience) stays on the axis; ROC's huge FPR denominator hides false alarms.

Q: What threshold would you ship? **A:** Show the PR curve as a menu to the cost owner; derive from expected cost if calibrated; two thresholds with a review band for fraud.

Q: Explain bias–variance and how to diagnose. **A:** Error = bias² + variance + noise; learning curves converged low → add capacity; big gap → data or regularisation.

Q: Why random over grid search? **A:** Same budget, a new value of every hyperparameter each trial, so important dimensions are explored more finely.

Q: How does Bayesian optimisation pick the next trial? **A:** Fits a surrogate of score vs hyperparameters and maximises an acquisition function (e.g. expected improvement) balancing predicted score and uncertainty.

Gotchas

- Accuracy on imbalanced data; precision without threshold and base rate; F1 across datasets with different base rates.
- Tuning the threshold or hyperparameters on the test set; reporting the best CV score as final.
- MAPE near y = 0; comparing RMSE across units; R^2 alone.
- NDCG with different gains/ideals (sklearn's `ndcg_score` builds the ideal only from passed items); evaluating only on items the old system showed.
- Calibrating on training data; isotonic on a few hundred rows; "high AUC = trustworthy probabilities".
- Preprocessing outside the CV loop; shuffled k-fold on time series.
- Adding data to a high-bias model; judging overfitting from training score alone.
- Early stopping on the test set; patience too small on noisy curves.

TL;DR

Baseline first; pick the metric from task + costs (PR-AUC for rare, NDCG for lists, MAE/RMSE + p90 for numbers); calibrate if probabilities matter; CV with the right splitter; test once; the A/B test decides.

7. Decision Trees and Ensembles

Trees and impurity, bagging and random forests, boosting (XGBoost/LightGBM/CatBoost), voting/stacking — and why trees still win on tables.

Must-know

- **Decision tree (CART)** — greedy top-down: try every feature/threshold, keep the split with the largest **impurity decrease** (Gini/entropy; variance for regression); leaves predict majority/mean. Readable, **scale-free**, non-linear, but **high variance**, axis-aligned, **can't extrapolate**. Control with `max_depth`, `min_samples_leaf`, `ccp_alpha` (cost-complexity pruning). Optimal tree is NP-hard → greedy.
- **Trees and bias–variance** — deep trees: low bias, high variance → average them (bagging/RF). Shallow trees: high bias → chain them (boosting).
- **Bagging** — deep trees on bootstrap samples, averaged → cuts variance. **Random forest** = bagging + a random feature subset per split to **de-correlate** trees. **~37%** rows out-of-bag per tree → free OOB score. More trees never overfit.
- **Feature importance** — MDI (impurity) is computed on training data, favours high-cardinality features, splits credit among correlated ones → use **permutation importance on held-out data** or SHAP.
- **Boosting** — shallow trees added sequentially, each fitting the **negative gradient** of the loss (residuals for squared error), scaled by a learning rate (shrinkage). Cuts bias; overfits with too many rounds → **early stopping**.
- **AdaBoost** — re-weights misclassified samples, exponential loss → sensitive to noise/outliers; a special case of gradient boosting.
- **Voting / stacking** — soft voting averages calibrated probabilities; stacking trains a meta-model on **out-of-fold** predictions.
- **Why trees beat NNs on tables** — irregular targets (sharp steps), many uninformative features, meaningful individual columns (not rotation-invariant), mixed types/missing values. NNs win on unstructured data, huge data, or end-to-end embeddings.

Key facts & numbers

Impurity	Formula	Pure	50/50 binary
Gini	$1 - \Sigma p_k^2$	0	0.5
Entropy	$-\Sigma p_k \log_2 p_k$	0	1.0 bit
Information gain	H(parent) − weighted avg H(children)		

	XGBoost	LightGBM	CatBoost	sklearn HGB
Tree growth	Depth-wise default	**Leaf-wise**	**Symmetric (oblivious)**	Leaf-wise (31 leaves)
Special	2nd-order (Newton) regularised objective; leaf $w^* = -G/(H + \lambda)$	GOSS + EFB for speed	**Ordered boosting**; best categorical handling	No extra dependency
Defaults	eta 0.3, max_depth 6	lr 0.1, num_leaves 31	—	lr 0.1, min_samples_leaf 20

	Bagging / RF	Boosting
Trees	Deep, parallel	Shallow, sequential
Reduces	Variance	Bias
More trees	Never hurts	Overfits → early stopping
Tuning	Little	Learning rate + tree size + early stopping

GBDT tuning order: lr 0.05–0.1 + early stopping → tree size and min leaf → subsampling → L1/L2 → lower lr and let early stopping add trees.

Code to remember

```
rf = RandomForestClassifier(n_estimators=500, oob_score=True, n_jobs=-1).fit(X_tr, y_tr)
print(rf.oob_score_)
gb = HistGradientBoostingClassifier(learning_rate=0.05, max_iter=2000,
        early_stopping=True, validation_fraction=0.1, n_iter_no_change=50).fit(X_tr, y_tr)
imp = permutation_importance(gb, X_val, y_val, n_repeats=10)   # held-out, not MDI
```

In the interview

Q: How does a tree choose a split? **A:** Tries every feature and threshold, keeps the largest impurity decrease; greedy; prune by depth/leaf size or `ccp_alpha`.

Q: Gini or entropy? **A:** Agree on almost all splits; Gini default because it's cheaper (no log). Not worth tuning.

Q: Feature 1000× larger in scale? **A:** Trees don't care — splits depend only on value order.

Q: Bagging vs random forest? **A:** RF adds a random feature subset per split to reduce correlation between trees.

Q: Does adding trees overfit a RF? **A:** No — test error converges; depth and leaf size control overfitting.

Q: Bagging vs boosting? **A:** Parallel deep trees averaged (variance) vs sequential shallow trees fixing errors (bias).

Q: Why a small learning rate in boosting? **A:** Each tree fixes only part of the error — regularises, generalises better, needs more trees.

Q: LightGBM vs XGBoost? **A:** LightGBM leaf-wise with GOSS/EFB for speed; XGBoost depth-wise by default with a regularised second-order objective; similar after tuning.

Q: Why out-of-fold predictions for stacking? **A:** In-sample predictions of an overfit model look perfect, so the meta-model would over-trust it.

Q: Why do trees beat neural nets on tabular data? **A:** Sharp thresholds, useless features ignored, columns respected, no scaling; GBDTs train in minutes on CPU and explain with SHAP.

Gotchas

- `max_depth=None` and reporting training accuracy (~100%, meaningless).
- Over-reading one tree's structure — resample and the top split may change.
- One-hot of thousands of levels into a tree → target/ordinal encoding or native categoricals.
- Treating MDI as causal or "true" importance.
- Boosting without early stopping; learning rate 1.0; depth-15 trees in boosting.
- Same set for early stopping and final reporting.

TL;DR

Tabular data → start with gradient-boosted trees (small lr + early stopping) and a random-forest baseline; explain with permutation importance or SHAP, not MDI.

8. k-NN, SVMs and Naive Bayes

Three classic classifiers — distance, margin and probability — and how to choose among the classic algorithms.

Must-know

- **k-NN** — store the training set; predict by vote/average of the k closest points. "Lazy": no training, prediction O(n·d) without an index. **Scale features**, choose k by CV (small k overfits, large k underfits), odd k for binary. Cosine for text/embeddings. Fails in high dimensions (**curse of dimensionality** — distances concentrate).
- **SVM** — the widest-street hyperplane: maximise margin **2/‖w‖**; only **support vectors** define it. **Soft margin C**: large C = tight fit, small C = regularise.
- **Kernel trick** — the dual needs only dot products, so replace φ(x)·φ(z) by K(x, z) computed in the original space. RBF default; **γ** = reach of each support vector (high γ → wiggly/overfit, low → smooth). Tune **C and γ jointly** on a log scale.
- **SVR** — fit inside an ε-tube; errors inside are free.
- **SVM vs logistic regression** — both linear with L2; SVM hinge loss ignores well-classified points (max margin); LR log-loss uses every point and gives probabilities.
- **Naive Bayes** — posterior ∝ prior × Π per-feature likelihoods, assuming features **independent given the class**. Multinomial (counts), Bernoulli (binary), Gaussian (continuous). **Laplace smoothing** (α = 1) + **log probabilities**. Works because classification only needs the argmax; probabilities are over-confident.
- **Generative vs discriminative** — NB models P(x|y)P(y) (learns fast from little data); LR models P(y|x) directly (usually wins with more data).
- **No free lunch** — no universal winner; baseline, then RF and GBDT, compare with CV.

Key facts & numbers

Algorithm	Scale?	Train	Predict	Best for	Avoid when
Logistic regression	Yes	Very fast	Very fast	Baseline, calibrated probs, sparse text	Complex interactions
k-NN	**Yes**	None	**Slow** O(nd)	Small low-dim data, similarity	Many features, big data, real-time
SVM (RBF)	**Yes**	Slow for large n (~$O(n^2d)$–$O(n^3d)$)	∝ support vectors	Small/medium data, curved boundaries	> ~100k rows, need probabilities
Linear SVM	Yes	Very fast	Very fast	Text at scale	Non-linear problems
Naive Bayes	No	**Fastest**	Very fast	Text, tiny data, streaming	Dependent features, need probabilities
Gradient boosting	No	Medium	Fast	**Best tabular accuracy**	Unstructured data

Loss	Formula
Hinge (SVM)	max(0, 1 − y·f(x)), y ∈ {−1, +1}
ε-insensitive (SVR)	max(0, |y − f(x)| − ε)

`SVC` defaults: RBF, C = 1, `gamma="scale"`. `KNeighbors*`: k = 5, Minkowski p = 2.

Code to remember

```
knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
svm = GridSearchCV(make_pipeline(StandardScaler(), SVC()),
                   {"svc__C": [0.1, 1, 10, 100], "svc__gamma": [0.001, 0.01, 0.1, 1]}, cv=5)
nb = make_pipeline(CountVectorizer(), MultinomialNB(alpha=1.0))     # text baseline
```

In the interview

Q: How does k-NN work and when does it fail? **A:** Vote/average of the k closest scaled points; slow O(nd) prediction; distances concentrate in high dimensions.

Q: Speed up k-NN? **A:** KD/ball trees in low dimensions, approximate NN (HNSW, FAISS) at scale, reduce dimensions first.

Q: Explain SVMs. **A:** Largest-margin hyperplane defined by support vectors; C trades width for violations; kernels give non-linear boundaries via dot products.

Q: What does γ do? **A:** Controls each support vector's reach: high → local and wiggly (overfit), low → smooth (underfit).

Q: Why is Naive Bayes naive, and why does it work? **A:** Assumes conditional independence; classification only needs the argmax right, so errors often cancel — but don't trust its probabilities.

Q: Laplace smoothing? **A:** Add α (usually 1) to every count so unseen words don't give zero probability.

Q: Which classic algorithm would you pick? **A:** Baseline (logistic/linear, NB for text) → RF → GBDT, compared with CV; deep learning for unstructured data.

Gotchas

- Not scaling for k-NN/SVM (or scaling on the full data before splitting).
- Even k in binary problems (ties); ID-like columns polluting distances.
- Euclidean on one-hot/text counts → Hamming or cosine.
- RBF on high-dimensional sparse text → linear kernel.
- `decision_function` values are not probabilities.
- NB: `alpha=0` → zero probabilities; multiplying raw probabilities → underflow; negative inputs to `MultinomialNB`.

TL;DR

k-NN = nearest examples (scale, tune k, low dimensions only); SVM = widest street, kernels bend it (scale, tune C and γ together); Naive Bayes = fast text baseline with over-confident probabilities.

9. Unsupervised Learning

Clustering (k-means, hierarchical, DBSCAN, GMM), dimensionality reduction (PCA, t-SNE/UMAP), anomaly detection, association rules, and judging results without labels.

Must-know

- **k-means (Lloyd)** — assign to nearest centroid, move centroids to means, repeat; minimises within-cluster squared distance; always converges, but to a **local** optimum → **k-means++** init (probability ∝ squared distance) + restarts. Assumes round, similar-sized clusters; scale features. Returns k groups even from uniform noise.
- **Agglomerative** — merge the closest clusters bottom-up; **linkage** (single chains → moons; complete; average; Ward) sets the shape; dendrogram; $O(n^2)$ memory → small data.
- **DBSCAN** — core point = ≥ min_samples neighbours within eps; clusters = chains of core points + borders; the rest is **noise (−1)**. Any shape, no k; eps from a **k-distance plot**; **HDBSCAN** for varying densities.
- **GMM + EM** — mixture of Gaussians; **E-step** = responsibilities (soft membership), **M-step** = weighted re-estimate. Soft, elliptical clusters; choose k with **BIC**. k-means = hard, spherical special case.

- **PCA** — centre (and scale), eigenvectors of the covariance (via SVD) ordered by eigenvalue = variance; keep ~95% explained variance. Linear, unsupervised, scale-sensitive; can drop low-variance but predictive directions.
- **t-SNE / UMAP** — non-linear 2-D maps preserving local neighbourhoods; **visualisation only**; cluster sizes and gaps are meaningless; seed/hyperparameter-dependent. UMAP is faster and can transform new points.
- **Anomaly detection** — robust z / IQR for one feature; **Isolation Forest** default (anomalies isolate in few random splits → short path); **LOF** for varying local density; **One-Class SVM** for novelty on clean data. Rank by score, judge with precision–recall.
- **Association rules** — support, confidence P(B|A), **lift** = confidence/supp(B) (> 1 = positive association). Apriori prunes: every subset of a frequent itemset is frequent.
- **Evaluation without labels** — internal metrics favour convex clusters; add external labels (ARI, NMI), stability across seeds, and downstream usefulness.

Key facts & numbers

Clusterer	Shape	Needs k?	Noise	Cost
k-means	Round, similar size	Yes (silhouette/elbow)	No	O(nkd)/iteration; MiniBatch for big data
Agglomerative	Set by linkage	Cut height	No	≥ $O(n^2)$
DBSCAN / HDBSCAN	Any	No	Yes	~O(n log n) with an index
GMM	Elliptical, soft	Yes (BIC)	No	$O(nkd^2)$ per EM step (full cov.)

Metric	Formula / meaning
Silhouette	s = (b − a)/max(a, b), higher better (a = mean intra-cluster distance, b = nearest other cluster)
Davies–Bouldin / Calinski–Harabasz	lower / higher better
ARI	0 random, 1 perfect (needs labels)
PCA via SVD	right singular vectors = directions; $\sigma_i^2/(n-1)$ = eigenvalues
Isolation Forest	`max_samples` 256 default

Code to remember

```
X_s = StandardScaler().fit_transform(X)
for k in range(2, 9):
    km = KMeans(k, n_init=10, random_state=0).fit(X_s)
    print(k, km.inertia_, silhouette_score(X_s, km.labels_))
pca = PCA(n_components=0.95).fit(X_s)            # keep 95% of variance
iso = IsolationForest(random_state=0).fit(X_s); scores = -iso.score_samples(X_s)
```

In the interview

Q: How does k-means work and how do you choose k? **A:** Assign/update until stable (local optimum); k-means++ and restarts; k by silhouette, elbow and domain sense.

Q: Why k-means++? **A:** Spreads initial centroids, giving faster, better, more consistent results.

Q: Linkage for moon-shaped clusters? **A:** Single linkage (it chains), if there's little noise between them.

Q: DBSCAN in 30 seconds. **A:** Dense regions become clusters, sparse points are noise; any shape, no k; struggles with mixed densities → HDBSCAN.

Q: GMM vs k-means? **A:** GMM gives soft, elliptical clusters with probabilities via EM; k-means is hard and spherical.

Q: What does PCA do? **A:** Rotates standardised data onto orthogonal max-variance directions; keep enough components for e.g. 95% variance.

Q: PCA vs t-SNE/UMAP? **A:** PCA is linear, global, for compression/pre-processing; t-SNE/UMAP are non-linear, local, for 2-D visualisation only.

Q: Why not plain z-scores for anomalies? **A:** The outlier inflates mean and std (masking), and it's univariate.

Q: Evaluate clustering without labels? **A:** Silhouette/DB/CH to compare settings (shape-biased), stability, and whether it helps the downstream task.

Gotchas

- Not scaling; trusting one random-init run.
- The elbow on data with no clusters.
- PCA without standardising (large-unit feature becomes PC1) or fitted before the split.
- Reading meaning into a 2-component PCA plot that explains 20% of variance.
- Reading cluster sizes or distances in t-SNE/UMAP plots.

TL;DR

k-means for round clusters, DBSCAN/HDBSCAN for odd shapes and noise, GMM for soft ellipses; PCA to compress, t-SNE/UMAP to look; Isolation Forest for anomalies; always sanity-check with stability and downstream value.

10. Neural Networks and Deep Learning

Perceptron to MLP, activations, losses, backpropagation, optimizers, initialisation, regularisation, training habits and the PyTorch loop.

Must-know

- **Neuron** — $z = w \cdot x + b$, $a = f(z)$. **MLP** — stacked layers; hidden layers learn features, depth composes them. Without nonlinearity the stack is **one linear map**. One perceptron can't learn XOR; one hidden layer can.
- **Universal approximation** — a one-hidden-layer net that approximates a continuous function *exists*; not that it's small or that training finds it.
- **Activations** — ReLU hidden default (can "die" → Leaky ReLU); GELU/SiLU in transformers; sigmoid/tanh saturate → outputs and RNN gates. Output: identity (regression), sigmoid (binary/multi-label), softmax (one of K).
- **Losses** — MSE/MAE/Huber (regression), BCE (binary/multi-label), cross-entropy (multi-class). Feed **logits** to `BCEWithLogitsLoss` / `CrossEntropyLoss`. Never train on accuracy (zero gradient).
- **Backpropagation** — chain rule from the loss backwards through the computational graph (**reverse mode**: one scalar output, millions of inputs → one sweep); ≈ 2–3× a forward pass; forward pass must store activations (memory ∝ batch × depth; gradient checkpointing trades compute for memory). Sigmoid+BCE / softmax+CE: $\partial L/\partial \text{logits} = p - y$.
- **Optimizers** — momentum (velocity), RMSProp (per-parameter RMS scaling), **Adam** (both + bias correction), **AdamW** (decoupled weight decay — not `Adam(weight_decay=…)`). Schedule: warmup + cosine decay. Warmup because early second-moment estimates are poor.
- **Vanishing/exploding gradients** — products of per-layer factors; fix with ReLU, **He** (ReLU) / **Xavier** (tanh) init, normalisation, **residual connections**, gradient clipping, gates.
- **Zero init** — symmetry never breaks; identical neurons get identical gradients.
- **Regularisation** — dropout (inverted: survivors × 1/(1−p); off at eval), weight decay, BatchNorm (running stats at eval; LayerNorm in transformers/RNNs), early stopping, augmentation; **more data beats all**.

- **Debugging** — look at raw data after the pipeline; initial loss ≈ **ln K**; **overfit one mini-batch**; LR range test; read both curves.

Key facts & numbers

Item	Value
Dense params	in × out + out; 256 → 128 = **32,896**
σ' max	0.25 → ten sigmoid layers ≤ $0.25^{10} \approx 9.5 \times 10^{-7}$
Softmax (2, 1, 0.1)	(0.659, 0.242, 0.099); subtract max first
Chance loss	ln K (ln 10 = 2.303)
Adam defaults	lr 1e-3, β (0.9, 0.999), ε 1e-8
AdamW	weight_decay 0.01 default; 0.01–0.1 typical
Momentum μ 0.9	≈ averaging ~10 steps
LR finder	pick ≈ 1/10 of the lr at the loss minimum
SGD batch heuristic	scale LR ~linearly with batch size + warmup

Optimizer	Update
SGD	$\theta \leftarrow \theta - \eta g$
Momentum	$v \leftarrow \mu v + g$; $\theta \leftarrow \theta - \eta v$
RMSProp	$s \leftarrow \rho s + (1-\rho)g^2$; $\theta \leftarrow \theta - \eta g/(\sqrt{s} + \varepsilon)$
Adam	$m \leftarrow \beta_1 m + (1-\beta_1)g$; $s \leftarrow \beta_2 s + (1-\beta_2)g^2$; $\hat{m} = m/(1-\beta_1^t)$, $\hat{s} = s/(1-\beta_2^t)$; $\theta \leftarrow \theta - \eta\hat{m}/(\sqrt{\hat{s}} + \varepsilon)$
AdamW	Adam + $\theta \leftarrow \theta - \eta\lambda\theta$

Init	Weight variance	For
Xavier/Glorot	2/(fan_in + fan_out)	tanh, sigmoid
He/Kaiming	2/fan_in (factor 2 because ReLU zeroes half)	ReLU
LeCun	1/fan_in	SELU

Curve	Cause	Try
Flat near ln K	lr too small / bug	Overfit one batch
NaN, spikes	lr too high, exploding grads, log(0)	Lower lr, clip, logit losses
Val ↓ then ↑	Overfitting	Regularise, early stop, more data
Both high	Underfitting	Bigger model, longer, higher lr

Code to remember

```
model.train()
for xb, yb in loader:                    # DataLoader(shuffle=True)
    logits = model(xb)
    loss = loss_fn(logits, yb)           # CrossEntropyLoss: logits + long class indices
    opt.zero_grad()                      # grads accumulate otherwise
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    opt.step(); sched.step()             # scheduler after optimizer
model.eval()
with torch.no_grad(): preds = model(X_val).argmax(1)
```

In the interview

Q: How does backprop work? **A:** Chain rule from the loss backwards, reusing stored activations; every gradient for about the cost of an extra forward pass; the optimizer then steps.

Q: Why non-linear activations? **A:** Otherwise any stack is one linear map. ReLU inside, sigmoid/softmax/identity out.

Q: Vanishing/exploding gradients? **A:** Products of many factors shrink/grow exponentially; ReLU, He/Xavier, normalisation, residuals, clipping, gating.

Q: SGD vs Adam vs AdamW? **A:** Adam scales steps per parameter with bias correction; AdamW decouples weight decay (default for transformers); SGD + momentum stays strong for CNNs.

Q: Why bias correction in Adam? **A:** m and s start at 0, biasing early averages to 0; dividing by $1 - \beta^t$ fixes it.

Q: Dropout at test time? **A:** Nothing — inverted dropout already rescaled in training.

Q: Why `zero_grad`? **A:** `.backward()` adds into `.grad`; without zeroing you sum batches (done on purpose for gradient accumulation).

Q: Loss won't go down — what do you do? **A:** Check data after the pipeline, chance-level first loss, overfit one batch, LR range test, gradient norms per layer.

Gotchas

- No activation between layers → linear model.
- `softmax` in the model + `CrossEntropyLoss` (already log-softmax) → trains poorly; probabilities into logit losses.
- Sigmoid in every hidden layer; ReLU output for a regression that needs negatives.
- Summing instead of averaging batch loss → effective lr depends on batch size.
- `scheduler.step()` before `optimizer.step()`.
- Evaluating in `train()` mode; augmenting validation data; early-stopping on training loss.

TL;DR

Neuron = weighted sum + nonlinearity; backprop = chain rule; AdamW + warmup-cosine; ReLU + He init + norm + residuals for stable depth; dropout/weight decay/early stopping/augmentation for generalisation; overfit one batch first.

11. CNNs and Computer Vision

Convolution and pooling, famous architectures, transfer learning, augmentation, detection and segmentation.

Must-know

- **Convolution** — slide shared k×k kernels (each spanning all input channels) over the image; weight sharing makes parameter count independent of image size; locality + sharing → **translation equivariance** (pooling adds approximate invariance).
- **Stride** downsamples; **padding** preserves size; **1×1 conv** mixes channels cheaply (bottlenecks).
- **Pooling** — 2×2 max, stride 2 halves H and W, no parameters; **global average pooling** before the head.
- **Architecture** — backbone of conv-BN-ReLU blocks (space shrinks, channels grow) + head (GAP + dense). Stacked 3×3 convs grow the receptive field cheaply.
- **ResNet** — y = x + F(x): identity easy to represent (F = 0) and a direct gradient path → fixes the degradation of deep plain nets.
- **ViT** — transformer on image patches; lacks CNN inductive biases (locality, weight sharing) → needs large-scale pretraining.
- **Transfer learning** — pretrained backbone + new head; small data → freeze and train the head; more/different data → fine-tune (top blocks first) with ~10× smaller lr; preprocess exactly like pretraining. Early layers (edges/textures) transfer best.

- **Augmentation** — random label-preserving transforms (crops, flips, colour jitter, rotations, Mixup/CutMix, RandAugment) on **training only**; for detection/segmentation transform boxes and masks too.
- **Tasks** — classification (label), detection (boxes + class; **IoU**, **NMS**), semantic segmentation (class per pixel; **U-Net** encoder–decoder with skip connections), instance segmentation (mask per object; Mask R-CNN, SAM).
- **Detectors** — two-stage Faster R-CNN (propose then classify) vs one-stage **YOLO** (one pass over a grid → real time). Judge by **mAP**, not accuracy.

Key facts & numbers

Quantity	Formula	Example
Output size	⌊(n + 2p − k)/s⌋ + 1	224, k 7, s 2, p 3 → 112
Conv weights	k·k·C_in·C_out + C_out	`Conv2d(3, 64, 7)` → 9,472; `Conv2d(64,128,3)` → 73,856
Conv vs dense	`Conv2d(3,16,3)` 448 vs dense on 32×32×3	49,168
Receptive field	three 3×3 = one 7×7	27C² vs 49C² weights
Depthwise-separable	per-channel conv + 1×1	≈ 8–9× less compute (MobileNet)
ViT tokens	(H/P)·(W/P)	224, P 16 → 196
IoU	intersection/union	VOC mAP@0.5; COCO 0.5–0.95
Layout	PyTorch N×C×H×W; TF/NumPy N×H×W×C	`permute(0, 3, 1, 2)`

Timeline: LeNet-5 (1998) → **AlexNet** (2012; ReLU, dropout, GPUs; top-5 15.3%) → VGG (2014; 3×3 stacks) → Inception (2014; 1×1, parallel branches, GAP) → **ResNet** (2015; skips) → MobileNet (2017) → EfficientNet (2019; compound scaling depth/width/resolution) → ViT (2020) → ConvNeXt (2022).

Code to remember

```
m = torchvision.models.resnet50(weights="IMAGENET1K_V2")
for p in m.parameters(): p.requires_grad = False        # freeze backbone
m.fc = nn.Linear(m.fc.in_features, n_classes)           # new head
opt = torch.optim.AdamW(m.fc.parameters(), lr=1e-3)
# later: unfreeze top blocks, lr ~10x smaller for the backbone
```

In the interview

Q: Why CNNs for images? **A:** Weight sharing and local connectivity → far fewer parameters and translation equivariance; stacked 3×3 + downsampling grow the receptive field; ResNet skips make depth trainable.

Q: Output of 224×224×3 through `Conv2d(3, 64, 7, stride=2, padding=3)`? **A:** 64 × 112 × 112; 9,472 weights.

Q: Is convolution translation invariant? **A:** Equivariant; pooling/GAP add approximate invariance.

Q: Why residual connections? **A:** Identity is easy (F = 0) and gradients get a direct path.

Q: Why does ViT need more data? **A:** No built-in locality/weight-sharing biases — it must learn them.

Q: Transfer learning recipe? **A:** Pretrained backbone, new head, freeze then fine-tune with a small lr, preprocess like pretraining; biggest gains with scarce labels.

Q: Why NMS? **A:** Detectors emit many overlapping boxes per object; keep the most confident, drop overlaps above an IoU threshold, repeat.

Q: Semantic vs instance segmentation? **A:** Class per pixel (all people = "person") vs separate masks per object.

Gotchas

- Mixing N×C×H×W and N×H×W×C; missing channel dim for grayscale.
- Computing the flatten size by hand → pass a dummy tensor or use `nn.LazyLinear`.
- Fine-tuning everything with a large lr on small data → catastrophic forgetting.
- BatchNorm updating running stats on tiny fine-tuning batches.
- Label-changing augmentations (flipping digits, mirrored road signs); augmenting val/test.
- Judging a detector by accuracy instead of mAP.

TL;DR

Shared kernels + pooling + residuals; start from a pretrained backbone, fine-tune with augmentation, and pick the head by task (classify, YOLO/Faster R-CNN detect, U-Net segment).

12. Sequences: RNNs, LSTMs and Attention

RNNs and vanishing gradients, LSTM/GRU, seq2seq, attention and the transformer, tokenization, word embeddings, and time-series forecasting.

Must-know

- **RNN** — the same cell at every step carrying a hidden state; unroll + **BPTT**; fixed parameter count for any length; can't parallelise over time (h_t needs h_{t-1}).
- **Vanishing/exploding through time** — one Jacobian per step multiplied → exponential shrink/growth. Exploding → **clip**; vanishing → **gates** or **attention**.
- **LSTM** — cell state updated **additively** $c_t = f \odot c_{t-1} + i \odot \tilde{c}_t$ with forget/input/output gates → gradients survive many steps. Forget-gate bias 1 ($f \approx 0.73$) so memory persists early. **GRU** — update/reset gates, no separate cell, 3 vs 4 weight blocks, similar accuracy.
- **Seq2seq** — encoder → context vector → decoder; **teacher forcing** in training, greedy/**beam search** at inference (beam k = 1 = greedy). **Exposure bias** = train/test mismatch from teacher forcing. Fixed-size bottleneck → attention.
- **Attention** — score a query against every key, softmax, weighted sum of values: **$\text{softmax}(QK^T/\sqrt{d_k})V$**. $\sqrt{d_k}$ keeps scores at unit variance so softmax doesn't saturate. Additive (Bahdanau, small MLP) vs dot-product (Luong/transformers, faster).
- **Transformer** — embeddings + positional encoding + N × (multi-head self-attention + position-wise FFN, each with residual + LayerNorm). Encoder bidirectional; decoder **causal mask** + cross-attention. Parallel, O(1) path between tokens, $O(n^2)$ cost.
- **Why multiple heads** — each attends in a different learned subspace; same total cost as one full-width head. **Why positional encodings** — self-attention is permutation-equivariant ("dog bites man" = "man bites dog").
- **Tokenization** — normalise → tokenize → vocabulary (`<pad>`, `<unk>`, `<sos>`, `<eos>`) → ids → pad + mask. Subword (BPE: merge most frequent adjacent pair; WordPiece; SentencePiece) → no OOV.
- **Word embeddings** — word2vec skip-gram (centre → context) / CBOW (context → centre) with **negative sampling** (k+1 binary classifications instead of a full softmax). Static: one vector per word; fastText uses character n-grams; transformers give contextual embeddings.
- **Time series** — past-only features (lags, **shifted** rolling stats, calendar), chronological/rolling-origin splits, beat **naive and seasonal-naive** baselines; ARIMA for single series; GBMs on lags for many series (model differences — trees can't extrapolate trends).

Key facts & numbers

Model	Equation / fact
RNN	$h_t = \tanh(W_{xh}\, x_t + W_{hh}\, h_{t-1} + b)$
LSTM	$f, i, o = \sigma(W[h_{t-1}, x_t] + b)$; $c_t = f \odot c_{t-1} + i \odot \tilde{c}_t$; $h_t = o \odot \tanh(c_t)$
GRU	$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$
Params RNN : GRU : LSTM	1 : 3 : 4
LSTM worked	c 2.0, f 0.9, i 0.2, c̃ 0.5 → c 1.9; o 0.7 → h = 0.669
Attention worked	Q = K = [[1,0],[0,1],[1,1]], V = [[1,2],[3,4],[5,6]] → query 1 weights (0.401, 0.198, 0.401), output [3.0, 4.0]
Positional encoding	PE(pos, 2i) = sin(pos/10000^(2i/d)), PE(pos, 2i+1) = cos(…); or learned / RoPE
Original transformer	6 + 6 layers, d_model 512, 8 heads, FFN ≈ 4× wider
Cost	attention $O(T^2 \cdot d)$ vs RNN $O(T \cdot d^2)$; path O(1) vs O(T); KV cache grows with context
Families	encoder-only (BERT), decoder-only (GPT, Claude, LLaMA), encoder–decoder (T5, Whisper)
ARIMA(p, d, q)	AR + differencing (the "I": removes trend) + MA

Code to remember

```
def attention(Q, K, V, mask=None):
    scores = Q @ K.transpose(-2, -1) / math.sqrt(Q.size(-1))
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float("-inf"))   # causal: upper triangle
    return torch.softmax(scores, dim=-1) @ V
lstm = nn.LSTM(input_size, hidden, num_layers=2, batch_first=True)  # (batch, time, feat)
out, (h_n, c_n) = lstm(x)
```

In the interview

Q: Why do RNN gradients vanish, and fixes? **A:** A product of per-step Jacobians; clip for exploding; LSTM/GRU additive paths or attention for vanishing.

Q: How does an LSTM fix it? **A:** Additive cell state controlled by sigmoid gates lets information and gradients flow across many steps.

Q: Explain attention. **A:** A learned weighted lookup: $\text{softmax}(QK^T/\sqrt{d_k})V$; in seq2seq each decoder step gets its own context.

Q: Why did transformers replace RNNs? **A:** Parallel over the sequence and any two positions connect in one step; cost is quadratic in length.

Q: Why a causal mask? **A:** Training processes all positions at once; the mask stops position i seeing the tokens it must predict.

Q: Why negative sampling? **A:** Full-vocabulary softmax per pair is expensive; k+1 binary tasks are cheap.

Q: Cross-validate a forecaster? **A:** Expanding/sliding windows — train on earlier, validate on the next period (`TimeSeriesSplit`).

Gotchas

- Missing `batch_first=True`; `out[:, -1]` on padded batches → pack or index real lengths.
- Not clipping RNN gradients.
- Random/K-fold splits on time series; rolling features including the current value.
- Skipping seasonal-naive baselines; trees on trending levels; one-step evaluation when the business needs 30 days ahead.

TL;DR

RNN → LSTM/GRU (additive memory) → seq2seq → attention softmax($QK^T/\sqrt{d_k}$)V → transformer (parallel, $O(n^2)$). Subword tokens, contextual embeddings; for forecasting, past-only features, chronological splits, seasonal-naive baselines.

13. Recommender Systems and Embeddings

Feedback types, similarity, content-based vs collaborative filtering, matrix factorisation, embeddings, two-tower retrieval, the funnel, ANN search, cold start, evaluation and popularity bias.

Must-know

- **Framing** — fill a huge sparse user–item matrix; optimise **top-K ranking** on an outcome closer to value than clicks (watch time, saves, purchases); state scale, latency, freshness, constraints first.
- **Explicit vs implicit feedback** — stars (scarce, real negatives) vs behaviour (abundant, positives only; **unobserved ≠ negative** → confidence weights + sampled negatives). Use implicit negatives you do have: early skips, "not interested".
- **Similarity** — cosine (direction), dot product (includes magnitude/popularity), adjusted cosine (subtract each user's mean), Jaccard for sets. Shrink similarities built on tiny overlaps. Train and serve with the **same** similarity.
- **Content-based vs collaborative filtering** — attributes (handles new items, over-specialises) vs co-behaviour (quality and surprise; cold start, popularity bias). **Item-based CF** beats user-based: more interactions per item, stabler, precomputable. Hybrids win.
- **Matrix factorisation** — $\hat{r} = \mu + b_u + b_i + p_u \cdot q_i$, trained on **observed** cells with L2 by SGD or **ALS** (fix one side, parallel ridge per user/item); weighted ALS for implicit. Not plain SVD (needs a full matrix; zeros = "hated").
- **Embeddings** — short learned vectors where near = related *for the training objective*; different training runs have different coordinate systems.
- **Two-tower** — user and item encoders meet only at a dot product → item vectors precomputed into an **ANN index**, user tower per request; in-batch softmax with temperature, **logQ correction**, a few hard negatives; ship tower + index as **one version**. Joint networks can't be precomputed → use them as the ranker.
- **Funnel** — retrieval 10^8 → 10^3 (recall; several retrievers) → ranking 10^3 → 10^2 (precision/NDCG, rich crosses, multi-task) → re-ranking → 10–50 (diversity via MMR, freshness, policy). Retrieval recall caps everything after.
- **ANN** — IVF (k-means lists; `nprobe` ↑ recall ↑), HNSW (layered graph; M, ef_search), PQ (compressed codes), Annoy trees. Report **recall vs exact search** with latency.
- **Cold start** — new users: popularity, onboarding, context, fast session updates; new items: content features + controlled exploration (bandits). No data must not mean never shown.
- **Evaluation** — time-based split, full-catalogue ranking (sampled metrics mislead), recall@k / NDCG / MAP + coverage, diversity, novelty, vs a **popularity baseline**; then A/B or interleaving.
- **Popularity bias** — popular items get more data → feedback loop. Measure (coverage, Gini of exposure), correct (logQ, IPS), explore, re-rank. Calibrate, don't eliminate.

Key facts & numbers

Formula	Meaning
Item-based CF	$\hat{r}(u,i) = \text{mean}(u) + \Sigma_j\, S(i,j)(r(u,j) - \text{mean}(u)) \,/\, \Sigma_j \lvert S(i,j)\rvert$
MF SGD	$\text{err} = r - \hat{r}$; $p_u \leftarrow p_u + \eta(\text{err}\cdot q_i - \lambda p_u)$
MF loss	$\Sigma(r - \hat{r})^2 + \lambda(\lVert p\rVert^2 + \lVert q\rVert^2 + b^2)$ over observed cells
Implicit confidence	$c = 1 + \alpha \cdot \text{count}$
Unit vectors	$\lVert a - b\rVert^2 = 2 - 2\cos$
InfoNCE / sampled softmax	$-\ln[e^{(s^+/\tau)}/\Sigma e^{(s/\tau)}]$, logits − log Q; $\tau \approx 0.05$–0.1
MMR	$\text{argmax}\ [\lambda\cdot\text{rel}(j) - (1-\lambda)\cdot\max_s \text{sim}(j, s)]$
Vector memory	N × dims × 4 B: 50 M × 128 → 25.6 GB

Code to remember

```python
# MF by SGD on observed (u, i, r) triples
for u, i, r in ratings:
    err = r - (mu + bu[u] + bi[i] + P[u] @ Q[i])
    bu[u] += lr * (err - reg * bu[u]); bi[i] += lr * (err - reg * bi[i])
    P[u], Q[i] = P[u] + lr * (err * Q[i] - reg * P[u]), Q[i] + lr * (err * P[u] - reg * Q[i])
# two-tower in-batch loss: logits = U @ I.T / tau - log(q_item); loss = CE(logits, arange(B))
```

In the interview

Q: Content-based vs CF, and cold start? **A:** Attributes vs co-behaviour; CF finds quality and surprise but fails on new users/items → popularity, onboarding, content features, exploration.

Q: Why item-based over user-based CF? **A:** More interactions per item, stabler similarities, precomputable offline; user–user over 100 M users is $O(U^2)$.

Q: Why not plain SVD for ratings? **A:** Needs a complete matrix; zero-imputing teaches "unseen = hated".

Q: Candidates from 100 M items in milliseconds? **A:** Two-tower with in-batch negatives + logQ; items in an ANN index; user vector per request; a few hundred to the ranker; tower + index versioned together.

Q: Where does diversity go? **A:** Re-ranking (MMR or quotas), because the ranker scores items one at a time.

Q: How do you evaluate a recommender? **A:** Time split, recall@k for retrieval, NDCG/MAP for ranking over the full catalogue, beyond-accuracy metrics vs a popularity baseline; then A/B on the real objective.

Q: Why can offline and online disagree? **A:** Logs reflect the old policy and ignore presentation and long-term effects.

Gotchas

- Unobserved = full-weight dislike; optimising clicks (clickbait wins).
- No bias terms in MF; tuning k on the test set.
- Duplicate item in a batch as its own negative; no popularity correction; mismatched tower/index versions; future events in user features.
- Cosine index fed unnormalised vectors; latency without recall; filtering after ANN leaves too few results.

TL;DR

Top-K ranking on a meaningful outcome; MF as the baseline; two-tower + ANN for retrieval, a rich ranker, then re-ranking for diversity; time-split full-catalogue evaluation against popularity; handle cold start and feedback loops deliberately.

14. Reinforcement Learning

MDPs and Bellman equations, bandits, dynamic programming, Monte Carlo vs TD, Q-learning and SARSA, DQN, policy gradients and actor-critic, RLHF, and the pitfalls.

Must-know

- **RL** — an agent learns a policy by trial and error: state → action → reward + next state; maximise long-run discounted return. Feedback is a score (not the answer), often delayed, and the agent's behaviour decides what data it sees.
- **MDP** — (S, A, P(s'|s,a), R, γ); Markov: next state depends only on current state and action. γ sets the horizon ≈ 1/(1 − γ).
- **V(s)** — how good a state is; **Q(s, a)** — how good an action in a state is (needs no model to act); greedy on Q* is optimal.
- **Bellman** — value now = reward now + γ × value next (expectation form for evaluation, optimality form with max).
- **Exploration vs exploitation** — bandits isolate it: **ε-greedy** (random at rate ε — decay it), **UCB** (optimism bonus shrinking with tries), **Thompson** (sample from each arm's posterior, act greedily; picks an arm with the probability it's best). Contextual bandits (LinUCB, Thompson) = RL's biggest industrial success (headlines, thumbnails).
- **Bandit vs A/B test** — a bandit shifts traffic to winners during the test (less regret) at the cost of cleaner statistics.
- **Dynamic programming** — known model: value iteration (repeat the optimality backup) or policy iteration (evaluate, act greedily, repeat).
- **Monte Carlo vs TD** — MC updates toward the actual return (unbiased, noisy, waits for episode end); TD bootstraps toward r + γV(s') (lower variance, every step). The **TD error δ** drives most of RL.
- **Q-learning (off-policy)** — target r + γ·max Q(s',·) → learns the optimal greedy policy (risky cliff-edge path while exploring); overestimates → Double Q-learning. **SARSA (on-policy)** — target r + γQ(s', a') with the action actually taken → safer path.
- **DQN** — neural Q-function + **experience replay** (decorrelate, reuse) + **target network** (frozen copy synced every C steps); Double DQN: select with online, evaluate with target. Discrete actions only.
- **Policy gradients** — push up log π(a|s) in proportion to return: REINFORCE (noisy) → baseline/advantage A = G − V(s) (less variance, no bias) → actor-critic (TD error as advantage) → **PPO** (clipped probability ratio; default; on-policy so sample-hungry). For continuous/huge action spaces, stochastic policies.
- **RLHF** — SFT → reward model from preference pairs → PPO on reward − β·KL(π ‖ π_SFT). DPO removes the RL loop; RLAIF uses AI feedback; GRPO uses group-mean baselines; verifiable-reward RL powers reasoning models.
- **Deadly triad** — function approximation + bootstrapping + off-policy → instability.

Key facts & numbers

Rule	Formula
Return	$G_t = \Sigma_k\ \gamma^k\ r_{t+k+1} = r_{t+1} + \gamma G_{t+1}$ (γ 0.9: horizon ≈ 10; $0.9^{10} \approx 0.35$)
Bellman optimality	$Q^*(s,a) = \Sigma_{s'}\ P[R + \gamma\ \max_{a'}\ Q^*(s',a')]$
UCB1	$\text{argmax}_a\ [Q(a) + c\sqrt{(\ln t\ /\ N_a)}]$, $c = \sqrt{2}$ in theory
Thompson (Bernoulli)	$\theta_a \sim \text{Beta}(\text{wins} + 1, \text{losses} + 1)$, pull argmax
MC	$V(s) \leftarrow V(s) + \alpha[G_t - V(s)]$
TD(0)	$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$; $\delta = r + \gamma V(s') - V(s)$
Q-learning	$Q \leftarrow Q + \alpha[r + \gamma\ \max_{a'}\ Q(s',a') - Q(s,a)]$
SARSA	$Q \leftarrow Q + \alpha[r + \gamma Q(s',a') - Q(s,a)]$
DQN loss	$\text{Huber}(r + \gamma\ \max_{a'}\ Q(s',a';\ \theta^-) - Q(s,a;\ \theta))$
REINFORCE	$\theta \leftarrow \theta + \alpha \cdot G_t \cdot \nabla \log \pi(a_t \mid s_t)$
Reward hacking	a re-collectable +3/step is worth $3/(1 - 0.9) = 30$

Code to remember

```
a = rng.integers(n_actions) if rng.random() < eps else rng.choice(np.flatnonzero(Q[s] == Q[s].max()))
s2, r, done = env.step(a)
target = r + gamma * Q[s2].max() * (not done)     # SARSA: Q[s2, a2]
Q[s, a] += alpha * (target - Q[s, a])
```

In the interview

Q: Explore vs exploit — methods? **A:** ε-greedy (random), UCB (optimism bonus), Thompson (posterior sampling); contextual bandits in production.

Q: Q-learning vs SARSA? **A:** Off-policy max target → optimal policy; on-policy actual-next-action target → learns what it does, safer while exploring.

Q: Why does Q-learning/DQN overestimate? **A:** Max over noisy estimates is biased upward; Double Q/DQN separates selection and evaluation.

Q: What makes DQN stable? **A:** Replay buffer (breaks correlation, reuses data) and a target network (stops the target moving).

Q: Why policy gradients over DQN? **A:** Continuous or huge action spaces and stochastic policies — max over continuous actions isn't practical.

Q: Explain RLHF. **A:** Supervised fine-tune, train a reward model on human preference pairs, optimise with PPO under a KL leash to the SFT model; DPO skips RL.

Q: When would you use RL, and what goes wrong? **A:** Sequential decisions that change future states with a simulator or safe experimentation; else supervised or a bandit. Risks: reward hacking, sample inefficiency, sparse rewards, seed instability, sim-to-real gap.

Gotchas

- Fixed ε forever; ignoring non-stationarity (use a sliding window/constant step); calling a winner from a few pulls.
- Bootstrapping from a terminal state (use the `(not done)` factor); time-limit truncation isn't a true terminal.
- Tie-breaking always to action 0 on an all-zero Q → break ties randomly.
- Tabular methods on large state spaces → function approximation.

TL;DR

Bellman: value = reward + γ × next value. Bandits for explore/exploit; Q-learning (off-policy) vs SARSA (on-policy); DQN = replay + target net; policy gradients → actor-critic → PPO; RLHF = SFT → reward model → PPO with KL.

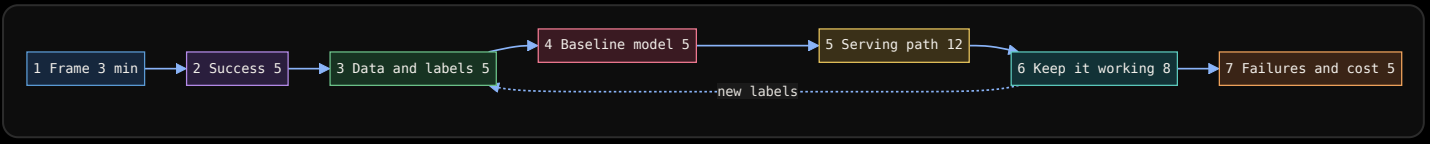
15. ML System Design: The Interview Framework

How an ML design round differs from classic system design, the three question families, the seven-step framework, napkin math and scoping a vague prompt.

Must-know

- **What changes with a model** — four classic assumptions break: correctness is a **spectrum** (need quality metrics); behaviour depends on **data** (drift); **compute is not cheap** (model calls cost money); **failures are silent** (a wrong answer is HTTP 200). Ask: *who is hurt by a wrong output, and what limits the damage?*
- **Five question forms** — known ML product, GenAI product, add AI to an existing system, design an agent, critique/scale an AI design. Ask in minute one: *go deep on model development or on the serving system?*
- **Graded** — requirements discipline, a **complete loop** (not just the request path), cost/latency realism, failure honesty. Secondary: deriving backprop, newest-model trivia.
- **Three families (output test)** — **predictive** (score/label: data, features, funnel, metrics, feedback), **generative** (content: retrieval/context, cost, output evaluation, safety), **agentic** (actions: tool permissions, step/cost limits, stop conditions, approvals). Mixed products: split into subsystems, go deep on the riskiest.
- **Build / host / API** — start with the simplest option meeting requirements; train your own when your interaction data *is* the product (ranking, fraud, recs).
- **Rules of ML** — #1 don't be afraid to launch without ML; #2 metrics first; #4 simple first model, get infrastructure right; #13 simple, observable, attributable first objective.
- **Business metric → ML objective** — frame input, output, ML category and the trained objective; name the proxy risk and its guardrail. Keep **product knobs** (thresholds, ETA buffers) outside the model.
- **Baseline first** — validates the pipeline, gives a number to beat, sometimes good enough; state the upgrade trigger.
- **Four serving questions** — offline vs online? where are caches (and permission rules)? which requests take a cheaper path? what if the model is slow/down (timeouts, fallback)?
- **Offline vs online evaluation** — offline filters candidates (cheap, proxy blind spots); online decides the launch (real traffic, needs power).

Key facts & numbers



Step	Say out loud
1. Frame	Family; the known system it resembles; scope in/out; ML task: input, output, category, **ML objective**
2. Success	Online business metric + offline ML metric + guardrails; latency p95/p99; scale (QPS, peak); cost per request or per completed task
3. Data	Sources; **label source and delay**; freshness (batch / streaming / request-time); feature groups; leakage and privacy
4. Model	Baseline (popularity, rules, LR) → GBDT → deep (two-tower, multi-task); name the upgrade trigger
5. Serving	Offline vs per request; caches with permissions; cheaper paths; timeouts and fallbacks; funnel if catalogue is big
6. Keep it working	Log inputs/outputs/versions → join outcomes → monitor 4 layers → gated retraining → shadow → canary → ramp
7. Failures & cost	Specific failures with controls; cost = requests × share × calls × price, in the same unit as the benefit

Business goal	ML objective	Why not the obvious one
Time spent	P(watch ≥ X s) or expected watch time	Raw clicks reward clickbait
Fraud losses	P(fraud) + cost-based thresholds	Accuracy meaningless at 0.2% fraud
Delivery promise	Minutes to delivery + product-owned buffer	Unbiased model is late ~half the time
Search revenue	P(purchase | query, item)	Relevance-only may ignore conversion
Churn	P(cancel within 30 days)	Label arrives 30 days late

Napkin math (illustrative numbers from the book)

Quantity	Arithmetic	Result
QPS	requests/day ÷ 86,400; peak ≈ 2–3×	10 M/day → 116 avg, ~347 peak
Homepage recs	100 M DAU × 5 loads ÷ 86,400	≈ 5,800/s avg, ~17 K peak; × 500 candidates ≈ 8.7 M scores/s
Feed ranking	500 M DAU × 5 sessions × 3 calls ÷ 86,400	≈ 86.8 K/s; × 500 posts ≈ 43 M scores/s
Ads scoring	100 K req/s × 300 candidates	30 M scores/s in a 50–100 ms auction
Vector memory	N × dims × 4 bytes	50 M × 128 → 25.6 GB; 100 M × 512 → 204.8 GB; PQ 64 B → 6.4 GB
Batch precompute	users × items	25 M × 500 = 12.5 B rows; active 5 M → 2.5 B
LLM cost	10 M req/day × $0.01	$100 K/day ≈ $3 M/month
Support bot	500 K orders × 8% × 10 calls × $0.01	40 K conversations, $4 K/day; compare **per resolved issue** (resolve 50% → $2.10 vs $4.00 human)
Moderation	200 M uploads/day, 1% → heavy, 0.1% → humans	2 M heavy/day (≈ 23/s), 200 K human decisions/day
A/B size	$n \propto 1/\delta^2$	5% CTR, +1% relative → ~3.0 M users/arm; half the lift → ~4×
Latency budget	inference 1–10 ms, online store 2–10 ms, inline features 0–15 ms, routing 1–5 ms, logging 1–2 ms	
Fraud	10 K tx/s × 0.2%	~20 fraud attempts/s; ~10 ms model budget in ~50 ms auth

Code to remember

```python
def qps(per_day, peak=3.0):
    avg = per_day / 86_400
    return avg, avg * peak
def model_cost_per_day(req_per_day, share, calls_per_req, cost_per_call):
    return req_per_day * share * calls_per_req * cost_per_call   # + retrieval, retries, humans
```

In the interview

Q: What changes vs an ordinary backend? **A:** Quality metrics beside latency/uptime SLOs; a loop that notices quality slips and tests new versions with staged rollout; a latency/spend budget per call; safeguards for bad outputs (validation, fallbacks, human review).

Q: Unclear prompt — how do you start? **A:** Pin down whether it outputs a score, content or actions; say my read and confirm; for a vague "use AI" prompt, list options by family and pick on impact × feasibility × risk.

Q: Why not let an LLM rank the feed? **A:** Ignores personalisation data, latency and cost at catalogue scale; plausible only as a reranker on the top ~50, benchmarked against a learned ranker.

Q: Why a baseline if GBDT will win? **A:** Validates pipeline and logging, gives a number to beat, sometimes good enough.

Q: What's your ML objective? **A:** Name the target, show how it moves the business metric, name the proxy risk and the guardrail.

Q: How will you know it works after launch? **A:** Four monitoring layers, outcome join, per-segment quality, drift alerts that start investigations, gated retraining.

Gotchas

- "I'd use a transformer" in minute 2 → frame, success, data first.

- Proxy objective with no guardrail (clicks → clickbait).
- "We'll have labels" → source, delay, noise, who pays.
- Random split on time data; accuracy on imbalanced data.
- No fallback; only the request path drawn.
- Training/serving features computed differently → one definition, log serving features.
- Auto-retraining on every drift alert.
- Hand-waving cost; comparing one model call vs one human ticket.

TL;DR

Frame (3) → Success (5) → Data (5) → Baseline (5) → Serving (12) → Keep it working (8) → Failures & cost (5). Translate the business goal into an observable ML objective, beat a baseline, show the loop, put numbers on cost.

16. ML in Production

Training vs serving, labels, feature stores, batch/online/streaming inference, serving, offline metrics in production, drift and retraining, A/B tests and MLOps.

Must-know

- **Five pieces, two planes** — training pipeline, feature layer, inference path, serving platform, monitoring loop; **offline plane** vs **online plane**. Models flow down to serving; logs and outcomes flow back. Skew, staleness, drift and cost live at the boundary.
- **Training pipeline** — data prep with **point-in-time-correct joins** → train → evaluate vs production → **model registry** with lineage.
- **Training–serving skew** — kill it with one feature definition, as-of joins, and training on **logged serving features** (Rules of ML #29, #32, #37).
- **Labels** — outcomes (slow), behaviour (fast, biased), humans (costly, need QA); train on **matured** rows; downsample then recalibrate; split by time; minimise PII.
- **Feature groups** — entity (nightly batch), behavioural aggregates (streaming), request context (inline, budgeted), crosses. **Feature store** = offline point-in-time history for training + online latest values for serving, from one definition. Alert on **feature freshness**; consider feeding feature age.
- **Inference mode** — deciding question: *does it need request-time information?* Stable & reused → **batch** (lookup; a failed job = degraded, not down); request-dependent → **online**; event-driven async → **streaming**; big catalogues → **precompute-then-rerank**.
- **Serving** — stateless, versioned service loaded from the registry; itemised latency budget; batched feature reads, hot-entity caches; **CPU first**; distil/quantise/prune; timeouts + agreed **fallback**; fail-open vs fail-closed by risk; **model version logged per prediction**.
- **Release ladder** — offline gate (overall **and per segment**) → **shadow** (copied traffic, side effects off) → sticky **canary** → ramp with a pre-agreed abort rule; rehearsed rollback of model + features + config.
- **Offline metrics in production** — at the shipping threshold with the workload it creates, per cohort, vs naive and production baselines; the cost owner picks the threshold.
- **Drift** — **data drift** (inputs change; may not hurt), **label shift** (base rate changes), **concept drift** (input→outcome relationship changes; always needs new labels), or plain bugs.
- **Monitoring (4 layers)** — service health → input distributions (PSI, KS, **null rates**) → prediction distribution → outcomes as labels arrive. An alert starts an **investigation** (often a broken pipeline); never auto-promote.
- **A/B tests** — randomisation unit, one primary metric, vetoing guardrails, power (MDE, α, $1 - \beta$), whole weeks, no peeking (or sequential test), **sample-ratio-mismatch** check; interleaving, switchbacks, holdbacks, bandits for special cases.
- **Feedback loops** — the model shapes its own labels; log impressions and selection probabilities; small risk-approved exploration.

- **MLOps** — experiment tracking (code, params, **data hash**, metrics; MLflow, W&B), data/model versioning (DVC, lakeFS, Delta), data validation that fails the pipeline (TFDV, Great Expectations, Pandera), registry with stages and owners, orchestration (Airflow, Kubeflow, Dagster), CI with training–serving consistency tests, CD with shadow/canary/auto-rollback, **CT** (continuous training).

Key facts & numbers

Mode	Freshness	Cost	Failure	Use
Batch	Stale (hours)	Cheap per request	Serve yesterday's (degraded)	Stable, reused predictions
Online	Fresh, request context	Pay at peak	Immediate incident → fallback	Fraud at checkout, ETA, search
Streaming	Event-triggered	Medium	Lag → freshness alert	Async updates on events
Hybrid	Precompute candidates, rerank online	—	—	Feeds, large catalogues

Monitor	Rule
PSI = Σ(cur − ref)·ln(cur/ref)	< 0.1 stable, 0.1–0.25 watch, > 0.25 investigate
KS statistic D	max gap between CDFs
Downsample recalibration	p = wq / (wq + 1 − q)
A/B sample size	n ∝ 1/δ²; 5% CTR, +1% relative lift → ~3.0 M users/arm
Latency (small model)	inference 1–10 ms; batch KV lookup ~1 ms vs online feature compute 10–100 ms

Code to remember

```
def psi(ref, cur, bins=10):
    edges = np.quantile(ref, np.linspace(0, 1, bins + 1)); edges[0], edges[-1] = -np.inf, np.inf
    r = np.histogram(ref, edges)[0] / len(ref); c = np.histogram(cur, edges)[0] / len(cur)
    r, c = np.clip(r, 1e-6, None), np.clip(c, 1e-6, None)
    return np.sum((c - r) * np.log(c / r))
```

In the interview

Q: Walk me through the ML architecture. **A:** Offline job builds point-in-time examples, trains, compares with live, registers with lineage; online stateless service computes features from the same definitions, calls the model with timeout and fallback, logs features/output/version.

Q: Fraud outcomes take six weeks — what do you train on? **A:** Matured rows up to six weeks back plus provisional signals (rule hits, analyst verdicts); score-distribution monitors cover the outcome gap.

Q: Why not compute every feature per request? **A:** Too slow and costly; precompute what tolerates staleness, stream what must be current, inline only what can't be known sooner.

Q: A/B two models cheaply? **A:** Both versions in one service, sticky per-user router, version logged per prediction; shadow first.

Q: Quality dropped overnight — debug. **A:** Bottom up: pipeline freshness and null rates → prediction histogram → segment the drop → only then the model.

Q: Data drift vs concept drift? **A:** Inputs change vs the input→outcome relationship changes; the second always needs new labels.

Gotchas

- Random split of time-ordered data; rewriting features in another language for serving.
- No fallback → checkout hangs when the model is slow; no timeouts/autoscaling.
- 7-day aggregates computed per request; silent streaming staleness.
- No model version per prediction → A/B can't be attributed.

- Rollback = "redeploy the old file" after the feature schema changed.
- Auto-retrain and auto-promote on drift alerts.
- CI that never checks training and serving produce the same features.
- Test the infrastructure independently: push a trivial model through and confirm identical offline and online predictions (Rules of ML #5).

TL;DR

One feature definition, point-in-time joins, train on logged serving features; pick batch/online/streaming by the need for request-time data; shadow → canary → ramp with fallback; monitor health → inputs → predictions → outcomes and investigate before retraining.

17. ML Case Studies I: Recommendations, Feed, Search

Four retrieval-and-ranking designs worked through one template; each summarised here as task → features → model → metric → hardest issue.

Must-know

- **Case template** — requirements → ML framing → data/labels → features → baseline → advanced → offline metrics → online metrics → serving → scale → monitoring → follow-ups.
- **The funnel** — too many items to score carefully per request: **retrieve** (a few hundred from millions; graded on **recall** — anything missed is lost for good) → **rank** (rich model; top-of-list precision) → **re-rank / rules** (freshness, variety, policy) kept outside the model.
- **Not everything is a funnel** — fraud scores one transaction at a time; moderation is a **cascade** (cheap checks on everything, expensive checks only on flagged items).
- **Proxy trap** — clicks reward misleading thumbnails, watch time rewards long videos → combine several predicted outcomes and check the long-term metric (retention) in experiments.
- **Position bias** — clicks depend on where an item was shown; log positions, correct with an **examination model** or **IPW** (inverse propensity weighting), judge offline on graded human labels.
- **Multi-task + value formula** — one shared body, one head per event; the product-owned weighted sum (harm terms subtracted) is what's optimised; heads must be **calibrated** and units normalised; weights live outside the model.
- **Learning-to-rank** — pointwise (each label), pairwise (which of two is better; RankNet), listwise (optimise NDCG); **LambdaMART** (GBDT with NDCG-weighted pairs) is the industrial default.
- **Hybrid search retrieval** — lexical (inverted index + **BM25**: exact codes, brands; blind to paraphrase) ∪ semantic (two-tower + ANN: meaning; unreliable on exact codes).
- **Video search** — index what a video *says* (ASR transcript) and *shows* (CLIP-style frame vectors) because titles are often empty or misleading; fuse channels with RRF or, better, a learned ranker told which channels are missing.

Key facts & numbers

17.1 Video recommendations

Task	Retrieval + learning-to-rank; objective = engagement per impression (P(click), expected watch time / P(watch ≥ 30 s))
Features	Watch history + embeddings, session state (last 10 videos, seconds-fresh), item CTR/watch-rate aggregates, user×item embedding similarity, context
Model	Popularity + subscriptions baseline → multi-source retrieval (two-tower ANN, co-occurrence, trending/fresh) → GBDT → deep multi-task ranker (YouTube: separate candidate and ranking nets; shallow tower for position bias)
Metric	Offline recall@K (retrieval), AUC/NDCG (ranking), time-based split / online watch time and return rate
Hardest issue	Cold start (both users and items) + position bias + clickbait proxy; judge new sources by **marginal recall**

17.2 Social feed ranking

Task	Multi-task classification/regression → value formula ranking over a social candidate pool (fan-in + spillover + suggested)
Features	Viewer–author closeness, author history and integrity, post content embeddings, streaming engagement (5 min, 1 h), session dedupe
Model	Reverse-chronological / LR P(like) baseline → shared-body multi-task net (MMoE when tasks conflict) + product-owned value formula + post-formula integrity layer
Metric	Per-head AUC, log loss, **calibration**, offline formula replay / sessions per user or worthwhileness + hide/report guardrails
Hardest issue	Balancing many goals without rewarding bait; ~86.8 K ranking calls/s, ~43 M scores/s

17.3 E-commerce search ranking

Task	Query understanding (spell-correct, synonyms, intent, attribute extraction → reliable ones become filters) + learning-to-rank
Features	Query–item match (BM25 per field, embedding similarity, attribute agreement), item quality (rating, sales, returns, stock), query class, capped personalisation
Model	BM25 + popularity + in-stock baseline → hybrid lexical ∪ semantic retrieval → L1 small GBDT → L2 LambdaMART/deep → optional cross-encoder on top ~20
Metric	NDCG@k on graded judgments, MRR for navigational queries / session success, revenue per search, reformulation and zero-result rates
Hardest issue	Position-biased clicks; revenue vs relevance (sponsored slots with a relevance floor)

17.4 Text-to-video search

Task	Cross-modal retrieval (text query → pictures + speech + text) + learning-to-rank on satisfied watch
Features	Per-field BM25 (title, description, ASR transcript, OCR), text–visual similarity (video and best segment), channel-availability flags, satisfied-watch rate, freshness, query class
Model	BM25 on metadata → + ASR (Whisper-style) → + CLIP-style text/frame towers (mean-pooled video vectors, segment vectors for moments) → RRF → learned fusion L1/L2 ranker
Metric	Recall@K per channel + marginal recall, NDCG@10, moment accuracy / searches ending in a satisfied watch, quick returns
Hardest issue	Indexing dominates cost (1 M uploads/day × 300 frames ≈ 3,500 encodings/s + ASR); missing/misleading text; segment vectors multiply the index

NDCG worked (exponential gain)	**Value**
Relevances [3, 0, 2]: DCG = 7/1 + 0 + 3/2	8.5
Ideal [3, 2, 0]: IDCG = 7 + 3/1.585	8.89
NDCG@3	≈ 0.956

Vector memory for video recs: 50 M × 128 × 4 B = 25.6 GB. Homepage: 100 M DAU × 5 loads ≈ 5,800 req/s avg, ~17 K peak.

Code to remember

```
# IPW for position-biased clicks: weight each click by 1 / P(examined at its position)
w = clicks / propensity[position]           # propensity from randomised swaps / examination model
score = (w * relevance_label).sum()
# value formula (weights owned by product, heads calibrated, dwell rescaled)
value = w_like*p_like + w_comment*p_comment + w_dwell*dwell_norm - w_hide*p_hide - w_report*p_report
```

In the interview

Q: Why not one end-to-end model for recommendations? **A:** Anything smart enough to order well is too expensive to run 50 M times per page; stages each with their own yardstick fix that.

Q: Recall looks fine but users are bored. **A:** Check re-ranking and per-segment objectives — diversity may have collapsed or the proxy over-rewards short-term clicks; watch the week-by-week category mix.

Q: New candidate source — include it? **A:** Measure marginal recall (items only it finds), then shadow, then a small experiment slice.

Q: Defend ranking over chronological. **A:** Newest-first buries valued posts for heavy followers; test that ranking improves interaction and satisfaction; offer a newest-first tab and "why am I seeing this".

Q: Who sets feed weights? **A:** Offline sweeps propose, A/B tests judge against primary metric and guardrails, product signs off — weights are policy.

Q: Search returns nothing? **A:** Diagnose no inventory vs vocabulary mismatch; relax uncertain inferred filters (never explicit ones), spelling variants, semantic-only retry, category browse; log it as a health metric.

Q: Video search: why not just titles? **A:** Many titles are empty, foreign or clickbait; index speech (ASR) and frames (CLIP-style), and give the ranker availability flags so a missing channel isn't read as a bad match.

Gotchas

- Opening with an architecture instead of "50 M items vs 200 ms forces a funnel".
- Treating clicks as ground truth; filtering hides/reports out as noise.
- Hiding diversity, policy or integrity inside the model instead of a separate layer.
- Rebuilding the whole funnel for the feed question — say "it transfers" and move to what's new.

- Jumping straight to embedding retrieval for search; treating the query as an opaque string; ignoring sponsored results.
- Video search on titles only; treating a missing transcript as a low score; one vector per video for moment queries.

TL;DR

Recs: multi-source retrieval judged by marginal recall, rich ranker, rules outside. Feed: calibrated multi-task heads + product-owned value formula + integrity layer. Search: query understanding + hybrid BM25/semantic retrieval + LambdaMART, with position-bias correction. Video search: index speech and frames, fuse channels with availability-aware ranking, segment vectors for moments.

18. ML Case Studies II: Ads, Fraud, Moderation, PYMK, Music, Visual Search, Blurring, Events, Similar Listings

Nine more designs with new twists — priced probabilities, adversaries, cascades, two-sided graphs, sessions, perishable indexes, privacy-critical detection, all-cold catalogues and session embeddings — plus the patterns that repeat across all thirteen.

Must-know

- **Ads** — the probability is *priced*: rank by **bid × calibrated pCTR**; AUC checks ordering only, so calibrate (Platt/isotonic), correct for negative downsampling and check **per segment**.
- **Fraud** — an **adaptive adversary**; keep the score separate from the **action policy** (approve / step-up or review / decline via two cost-derived thresholds); rules and model together on purpose — every rule has an owner, a dashboard and an **expiry date**.
- **Moderation** — a **cascade** with an **action ladder**; measure **prevalence of violating views**; flag rate = compute bill, review threshold = headcount.
- **PYMK / music / visual search** — two-sided score P(send) × P(accept | sent) with privacy **fail-closed**; audio embeddings for zero-play tracks and session-driven radio; a shared image–text space whose index **churns** (embed at creation, tombstone on sale, version encoder + index together).
- **Blurring / events / similar listings** — recall-first detection judged per person across views and per pixel, failing closed; every event is new, so GBDT on content, place, time and friends with point-in-time features; skip-gram listing vectors from click sessions with same-market negatives and averaged vectors for new listings.
- **Patterns across all thirteen** — funnels and cascades; rules outside the model; biased and late labels; two-sided or multi-goal objectives; fail-safe defaults decided in advance.

Key facts & numbers

18.1 Ad click prediction

Task	Binary classification with **calibrated** pCTR (+ pCVR heads) feeding an auction; minimise log loss
Features	Sparse user/context/ad IDs (embeddings or hashing), crosses, advertiser history, streaming counters
Model	LR on hashed crosses (FTRL) → GBDT-as-features + LR → FM / Wide & Deep / DeepFM / DCN; ESMM pCTCVR = pCTR × pCVR; calibration layer
Metric	Log loss, normalised entropy, AUC, **ECE per segment** / RPM, advertiser ROI
Hardest issue	Per-segment calibration; delayed conversions; 100 K QPS × 300 candidates = 30 M scores/s in 50–100 ms

18.2 Payment fraud

Task	Imbalanced binary classification + cost-based three-way action policy
Features	**Velocity** (tx per card in 10 min/1 h), impossible travel, device/session, **graph** links (rings), merchant rate from past windows only
Model	Rules + LR → GBDT + graph features (GNN for rings), optional isolation-forest score
Metric	**PR-AUC**, recall at fixed FPR, precision at review capacity, expected $ loss on a time-based backtest with mature labels / fraud bps, false-decline rate
Hardest issue	Adaptive adversary; labels weeks late and **censored** by blocks; 10 K tx/s, ~10 ms model budget

18.3 Harmful content

Task	Multi-label multimodal classification run as a cost cascade, then an action ladder
Features	Text/image/frame/OCR/ASR/audio embeddings, exact + perceptual hashes, account history, share velocity, reputation-weighted reports
Model	Hashes + rules → fast per-modality classifiers → heavy multimodal/LLM → humans; distil heavy into fast
Metric	Per-category P/R at thresholds on fully reviewed sets, whole-cascade recall, hash recall under mutation / violating-view prevalence, false-removal & overturn rates
Hardest issue	Harm both ways at scale; reviewer capacity (200 M uploads/day → 2 M heavy, 200 K human); evasion

18.4 People You May Know

Task	Link prediction + two-sided ranking; score = P(send) × P(accept | sent)
Features	Mutual count, Adamic-Adar Σ 1/log(deg), shared affiliations with overlapping dates, accept history, graph embeddings, impression count
Model	Mutual-count sort → GBDT / two-head net over FoF + affiliations + contacts + ANN candidates
Metric	Recall@K of future edges, per-head AUC + calibration / accepted invites per impression, "don't know" reports
Hardest issue	Two-sided objective; privacy fail-closed; bounded graph-scale batch; rich-get-richer

18.5 Music discovery

Task	Batch recommendation with novelty (weekly) + online sequential next-track (radio)
Features	Collaborative vectors, audio embeddings, metadata/playlist text, session context (seed, last N, skips)
Model	Cohort popularity → implicit ALS / two-tower + audio encoder mapped into CF space → GRU4Rec/SASRec for radio → ranker
Metric	Recall/Hit@K, MRR, coverage, artist Gini, cold-start slice / saves, skip rate, long-term repeat listening
Hardest issue	New-track cold start; skip-responsive radio; diversity and artist fairness

18.6 Visual search

Task	Representation (metric) learning + ANN retrieval + ranking
Features	Image/text embeddings; filters (category, price, location, live); ranker: similarity, price proximity, condition, seller rating, freshness
Model	Pretrained CLIP-style → contrastive fine-tune (InfoNCE, hard negatives) + GBDT ranker; detector crops the dominant item
Metric	NDCG@k and recall@k **per category**, ANN recall vs exact / contact-seller rate
Hardest issue	Index churn (sold = deleted); encoder upgrades needing re-embeds (100 M × 512 floats = 204.8 GB; PQ 64 B → 6.4 GB)

18.7 Privacy blurring in street imagery

Task	Batch object detection (faces, plates), recall-first; output padded, irreversible blur regions
Features	Overlapping perspective tiles, multi-scale (2× horizon band for tiny faces), neighbouring panoramas, geometry priors
Model	Off-the-shelf detectors → fine-tuned one-stage detector (YOLO/RetinaNet) at a low threshold → context classifier (cascade) → multi-view box propagation → human review
Metric	Recall by size/region/lighting, object-level recall across views, pixel coverage (leak if > 5% uncovered) / audited misses per 1,000 panoramas, report rate, time-to-fix
Hardest issue	Misses are privacy breaches; tiny faces; no A/B tests (audits + shadow); fail closed; ~14 GPUs for 5 M panoramas/day, archive re-processing is a separate budget

18.8 Event recommendation

Task	Pointwise LTR, P(register) (× P(attend)); every item is new and perishable, so content/context-based
Features	Topic affinity, distance/travel time, date fit, friends registered, organiser quality, momentum and remaining capacity (as of impression time)
Model	Nearby-popular rules → GBDT on features → geo+time filter + content-only two-tower retrieval + deep multi-task ranker; exploration quota
Metric	AUC, hit@k on events created after the training window / registrations per user, attendance, events with ≥ 1 registration
Hardest issue	All-cold catalogue (item-ID CF ≈ random); point-in-time leakage; delayed labels; two-sided organiser health

18.9 Similar listings (session embeddings)

Task	Item-to-item representation learning + per-market k-NN + ranking
Features	Session-embedding similarity, in-session vector, price ratio, distance, type/capacity match, availability, quality
Model	Attribute k-NN → skip-gram on click sessions (booked listing as global context, same-market negatives; Grbovic & Cheng, 2018) → GBDT ranker on bookings; new listings = mean of 3 similar nearby
Metric	Next-click and booking hit rate@K after filters / site-wide bookings, anchor-cannibalisation guardrail
Hardest issue	Learning "substitutes" without labels; cold start; availability filtering; retrained spaces rotate → version vectors + index together

Formula	Use
Downsampled CTR correction: q = p / (p + (1 − p)/w)	Ads (He et al., 2014)
Cost threshold: flag if p > C_FP/(C_FP + C_FN)	Fraud, moderation
Normalised entropy = log loss / log loss of predicting average CTR	Ads
ECE = size-weighted mean |mean prediction − observed rate| per bucket	Ads calibration

Code to remember

```
def decide(p, amount, low=0.02, high=0.6):     # thresholds from expected cost + review capacity
    if p < low:  return "approve"
    if p < high: return "step_up_or_review"
    return "decline"
# ads auction: rank = bid * calibrated_pctr ; recheck calibration per segment
```

In the interview

Q: Why does calibration matter for ads? **A:** The probability is multiplied by money in the auction; AUC can be excellent while every pCTR is 3× too high.

Q: Conversions are scarce — how does pCVR learn? **A:** Factor into click × conversion-given-click (ESMM) so each head uses the densest labels; correct for delay; priors for thin advertisers.

Q: Once the fraud model is strong, drop the rules? **A:** No — speed during attacks, hard constraints, readable logic; manage rules with metrics and expiry dates.

Q: Harmful content spreading fast right now? **A:** Share velocity is an input: fast spreaders jump to deeper checks and the queue head; each view tier triggers another pass.

Q: FoF for 900 M members? **A:** Top neighbours only, active members nightly, sampled celebrity lists, capped outputs — never the full two-hop closure.

Q: Visual search: why not classify category then search inside? **A:** Classifiers break on straddling items; embeddings degrade gently — use taxonomy as filter/feature.

Q: Blurring: can you guarantee 100%? **A:** No — low threshold + cascade + multi-view tracking + padding + human review, then audits and a fast report-and-fix path; state the audited miss rate.

Q: Why not matrix factorisation for events? **A:** Each event lives weeks with few registrations; score features that transfer (category, place, time, organiser, friends).

Q: Similar listings: why same-market negatives if you filter by market? **A:** Random negatives are almost all other-market, so the model spends capacity learning markets (the filter's job) instead of ordering within one.

Gotchas

- Ads: stopping at AUC; starting with relevance before eligibility (targeting, budget, policy); diagram ending at the winning ad.
- Fraud: one threshold; accuracy at 0.2%; merchant fraud rate computed with future data.
- Moderation: one big classifier; remove vs allow only; thresholds read straight off a PR curve.
- PYMK: optimising invites or acceptance alone; privacy filtering only at display; nightly-batch dismissals.
- Music: one serving mode for both products; CF only; engagement only.
- Visual search: static index; swapping encoders like a normal deploy; stopping at nearest neighbours.
- Blurring: threshold 0.5; per-view recall; exact (unpadded) boxes; failing open.
- Events: item-ID CF; momentum features computed at training time; ranking by popularity only.
- Similar listings: attributes only; random negatives only; optimising carousel clicks instead of site-wide bookings.

TL;DR

Ads = priced probabilities (calibrate per segment); fraud = adversary + two thresholds + rules and GBDT; moderation = cascade + prevalence of views + action ladder; PYMK = P(send) × P(accept) with privacy fail-closed; music = batch + session radio with audio embeddings; visual search = shared image–text space with a perishable, versioned index; blurring = recall per person and per pixel, fail closed; events = all-cold catalogue scored by content, place, time and friends; similar listings = skip-gram session embeddings with same-market negatives.

19. Responsible ML

Where bias comes from, fairness definitions and their conflict, measuring and mitigating bias, explainability, privacy, security, documentation and regulation.

Must-know

- **Sources of bias** — historical decisions, sampling/coverage gaps, measurement, **biased labels** (arrests, past hires, cost as proxies), aggregation, evaluation, deployment, **feedback loops**. No algorithm fixes a biased label; more of the same skewed data makes the model more confident in the skew.
- **Fairness families** — **independence** (demographic parity), **separation** (equal opportunity = equal TPR; equalized odds = equal TPR and FPR), **sufficiency** (predictive parity = equal PPV; calibration within groups). Also individual fairness and counterfactual fairness.
- **Impossibility** — with different base rates you can't have calibration *and* equal FPR *and* equal FNR unless the model is perfect (Kleinberg et al. 2016; Chouldechova 2017; COMPAS). Choose the criterion by harm, document why, report the rest.
- **Choosing a criterion** — hiring with discriminatory base rates → parity may be justified; loans/screening → equal opportunity; criminal justice → equalized odds; human-read risk scores → calibration.
- **Measuring** — on held-out data: per-group selection rate, TPR, FPR, PPV, calibration, the gaps, **group sizes and CIs**, intersectional groups; **proxy test** (predict the group from features). Removing the sensitive column isn't enough — keep it separately, access-controlled, for auditing.
- **Mitigation** — pre-processing (better data, **reweighing**), in-processing (fairness penalty/constraint; usually best trade-off), post-processing (group thresholds on **validation** data; uses the attribute at decision time — legal implications). Measure the accuracy cost; re-audit every criterion.
- **Explainability** — global: permutation importance on held-out data, PDP/ICE; local: **SHAP** (additive, axioms: efficiency, symmetry, dummy; TreeSHAP exact for trees), LIME (local linear surrogate; faster, less stable), counterfactuals. Correlational — for debugging and reason codes, never causal claims.
- **Privacy** — pseudonymised ≠ anonymous (quasi-identifiers link back); k-anonymity (+ l-diversity) resists linkage without a formal guarantee; **differential privacy** bounds any one person's influence (noise scale $\Delta f/\varepsilon$; budgets compose); DP-SGD; **federated learning** moves the model, not the data (+ secure aggregation, DP since updates leak).
- **Security** — evasion (adversarial examples: FGSM/PGD → adversarial training), poisoning/backdoors (provenance, anomaly checks), extraction and inversion via APIs (rate limits, auth, label-only outputs), membership inference (regularisation, DP), generative misuse.
- **Documentation** — **model card** (intended and out-of-scope use, data, per-group metrics, limitations) and **datasheet** (motivation, composition, collection, labelling, uses, maintenance), generated in the pipeline and versioned.
- **Regulation** — GDPR (lawful basis, minimisation, erasure, **Art. 22** safeguards for solely automated significant decisions); EU AI Act risk tiers (banned uses; **high-risk** incl. credit and hiring needs risk management, data governance, logging, human oversight; transparency duties for chatbots/deepfakes); US fair lending (adverse-action reasons), disparate impact, NYC bias audits (LL 144).

Key facts & numbers

Formula	Value / meaning
Disparate impact	P(Ŷ=1 | B)/P(Ŷ=1 | A); 0.35/0.50 = 0.70 < 0.8 (four-fifths rule) → warning
Chouldechova	FPR = p/(1−p) · (1−PPV)/PPV · (1−FNR)
Reweighing	w(g, y) = P(g)P(y)/P(g, y)
Fairness penalty	loss + λ·(mean score A − mean score B)²; larger λ → more parity, less accuracy
SHAP	contributions add up to f(x) − E[f]
ε-DP	P[M(D) ∈ O] ≤ e^ε · P[M(D') ∈ O]
Laplace noise	scale Δf/ε: count (Δf 1), ε 0.5 → 2; k queries at ε each cost kε
FGSM	x_adv = clip(x + ε·sign(∇ₓL), 0, 1)
FedAvg	data-size-weighted average of client weights

Code to remember

```
for g, idx in df.groupby("group").groups.items():
    yt, yp = y_true[idx], y_pred[idx]
    tn, fp, fn, tp = confusion_matrix(yt, yp, labels=[0, 1]).ravel()
    print(g, len(idx), "sel", yp.mean(), "TPR", tp/(tp+fn), "FPR", fp/(fp+tn), "PPV", tp/(tp+fp))
proxy_auc = cross_val_score(GradientBoostingClassifier(), X, group, scoring="roc_auc").mean()
```

In the interview

Q: Your model is 94% accurate — is it fair? **A:** Unknown until you slice: selection rate, TPR, FPR, PPV, calibration per group with CIs, plus a proxy check; criteria conflict when base rates differ.

Q: Where does bias come from? **A:** Mostly data — past decisions, thin coverage, proxy labels/features, feedback loops; audit coverage and per-group metrics and question the label.

Q: Which criterion for loans? **A:** Usually equal opportunity plus calibration checks, confirmed with legal and domain experts.

Q: SHAP vs LIME? **A:** SHAP has axioms and consistency (TreeSHAP exact); LIME is faster but unstable.

Q: Explain a decline to a customer? **A:** Top negative SHAP contributors mapped to plain-language reason codes, plus a counterfactual.

Q: Is hashing user IDs anonymisation? **A:** No — pseudonymisation; linkage re-identifies and GDPR still applies.

Q: Someone is scraping our model API? **A:** Extraction risk (enables white-box attacks): rate limits, auth, label-only responses, query-pattern monitoring, terms.

Q: Design a fair hiring screener. **A:** Question automating at all; label = job performance, not past hires; audit coverage; pick a criterion with legal; mitigate and re-audit; reasons to candidates; human review; bias audits; monitor per group.

Gotchas

- "Just maths" = neutral; only overall accuracy checked; one unnamed fairness number.
- Tiny groups without CIs; forgetting intersectional groups; auditing on training data; auditing once.
- Group thresholds fitted on the test set.
- Importance on training data; twin correlated features each looking unimportant; one LIME run as truth; explaining log-odds vs probability confusingly.
- DP without input clipping (unbounded sensitivity) or ignoring composition; raw PII in logs and prompts; keeping data forever.

- Robustness tested only with FGSM; gradient-masking "defences"; full probability vectors from public APIs.
- Model card written once, only aggregate metrics, no out-of-scope uses.

TL;DR

Bias comes from data and labels; pick a fairness criterion by harm (they conflict), audit per group with CIs and proxy tests, mitigate and re-audit; explain with SHAP/permutation importance (not causally); DP and federated learning for privacy; adversarial training, provenance and API hardening for security; document with model cards.

20. Cheat Sheets (and Glossary)

The book's one-page sheets condensed into the two tables you reach for most — every loss and "which algorithm for which problem" — plus the workflow. (The 348-term glossary is not repeated here.)

Must-know

- **Workflow** — frame (decision, metric, baseline) → data (labels, EDA, hold out test) → prepare (all inside a `Pipeline`) → train (simple first) → evaluate and tune (CV, test once) → deploy (pipeline as one artifact; shadow, canary) → monitor (drift, outcomes, retrain) → loop.
- **Pick output layer and loss together** — identity + MSE/MAE/Huber (regression); sigmoid + BCE (binary, multi-label); softmax + cross-entropy (one of K). Never train on accuracy.
- **Tabular → GBDT; unstructured → pretrained deep nets; text baseline → TF-IDF + LR; recs → popularity + item-CF → two-tower + ranker.**

Key facts & numbers

Loss functions

Loss	Formula (one example)	Used for	Note
MSE	$(\hat{y} - y)^2$	Regression, NN	Optimal constant = mean; MLE under Gaussian noise
MAE	$\lvert\hat{y} - y\rvert$	Robust regression	Optimal constant = median
Huber	$\frac{1}{2}e^2$ if $\lvert e\rvert \le \delta$, else $\delta(\lvert e\rvert - \frac{1}{2}\delta)$	Outliers; DQN	Quadratic near 0, linear far
Quantile (pinball)	$\max(\tau e, (\tau - 1)e)$, $e = y - \hat{y}$	Intervals, p90 forecasts	$\tau = 0.9$ → 90th percentile
Log loss / BCE	$-[y \ln p + (1-y) \ln(1-p)]$	LR, sigmoid outputs, multi-label	Gradient at logit = $p - y$; `BCEWithLogitsLoss`
Cross-entropy	$-\ln p_{true}$	Softmax classifiers, LMs	Uniform guess = $\ln K$; `CrossEntropyLoss` (logits + indices)
Hinge	$\max(0, 1 - y \cdot f(x))$	SVM	Zero beyond the margin
ε-insensitive	$\max(0, \lvert y - f(x)\rvert - \varepsilon)$	SVR	Errors inside the tube free
Exponential	$e^{-y \cdot f(x)}$	AdaBoost	Explains outlier sensitivity
MF regularised SE	$\Sigma(r - \hat{r})^2 + \lambda(\lVert p\rVert^2 + \lVert q\rVert^2 + b^2)$	Matrix factorisation	Observed cells only
InfoNCE / sampled softmax	$-\ln[e^{s^+/\tau}/\Sigma e^{s/\tau}]$ (− log Q)	Two-tower, CLIP	$\tau \approx 0.05$–0.1
Skip-gram neg. sampling	$-\ln \sigma(v \cdot u^+) - \Sigma \ln \sigma(-v \cdot u^-)$	word2vec, item2vec	Negatives $\propto$ freq$^{0.75}$
Ranking	pairwise (RankNet/LambdaMART), listwise (NDCG-like)	Learning-to-rank	Pointwise = each label independently
L2 / L1 penalty	$\lambda\Sigma w^2$ / $\lambda\Sigma\lvert w\rvert$	Any	L1 → zeros; AdamW = decoupled L2
TD target	$(r + \gamma \max Q(s',a'; \theta^-) - Q(s,a; \theta))^2$	DQN (Huber)	θ^- = target net
Policy gradient	$-\log \pi(a\lvert s) \cdot A_t$	REINFORCE, actor-critic	$A = G - V(s)$ or TD error
Reward model	$-\ln \sigma(s_{preferred} - s_{rejected})$	RLHF	Then PPO on reward − β·KL
Fairness penalty	loss + λ(mean score A − mean score B)²	In-processing mitigation	λ ↑ → parity ↑, accuracy ↓

Which algorithm for which problem

Problem	Start with	Then try	Avoid
Tabular, < ~1 M rows	Linear/logistic; dummy baseline	RF → **gradient boosting**	Deep nets on small tables; k-NN beyond ~20–50 features
Many categoricals	CatBoost / HGB native categoricals	Cross-fitted target encoding + GBDT	One-hot of 100 K levels for linear
Sparse text	TF-IDF + LR / linear SVM; NB for tiny data	Fine-tuned transformer	RBF SVM on sparse text
Images	Fine-tune pretrained CNN/ViT + augmentation	Bigger backbone, more data	Training from scratch on small data
Detection / segmentation	YOLO or Faster R-CNN; U-Net	DETR	Accuracy as the metric (use mAP)
Sequences / generation	Pretrained transformer	LSTM/GRU for small/streaming/on-device	Simple RNN for long dependencies
Time-series forecast	Naive, seasonal-naive; ETS/ARIMA	Global GBDT on lags (model differences)	Random splits; trees on trending levels
Round clusters	k-means (k-means++, scaled)	GMM, agglomerative	k-means on moons/rings
Odd shapes / noise	DBSCAN / HDBSCAN	—	One eps for mixed densities
Compression	PCA (standardise, ~95%)	Kernel PCA, TruncatedSVD	PCA for non-linear visualisation
Unlabelled anomalies	Robust z / IQR; Isolation Forest	LOF, One-Class SVM, autoencoders	Plain z on skewed data
Rare-event (fraud)	Rules + LR → GBDT with velocity/graph	GNN for rings	Accuracy; single threshold
Recommendations	Popularity + item-CF	Implicit ALS → two-tower + ranker	Pure CF for always-new catalogues
Search ranking	BM25 + popularity	Hybrid retrieval, LambdaMART, cross-encoder top-k	Raw position-biased clicks
Few options, live feedback	A/B test	Thompson, UCB, LinUCB	Fixed ε forever
Sequential decisions + simulator	Value iteration / Q-learning	DQN, PPO/SAC	Online RL on live money or people
Must explain every decision	Linear/logistic, small tree, GAM	GBDT + SHAP reason codes	Black box without oversight

Code to remember

```
nn.MSELoss(); nn.HuberLoss(delta=1.0)                # regression
nn.BCEWithLogitsLoss(pos_weight=w)                   # binary / multi-label, logits in
nn.CrossEntropyLoss(weight=class_w)                  # one of K, logits + long indices
```

In the interview

Q: Which metric for rare positives? **A:** PR-AUC plus precision/recall at the operating threshold and expected cost; not accuracy, not ROC-AUC alone.

Q: Which loss for a p90 delivery promise? **A:** Quantile (pinball) loss with $\tau = 0.9$ — or a regressor plus a product-owned buffer.

Gotchas

- Quoting a formula without the baseline or unit next to it.
- Softmax for multi-label; accuracy as a training objective.

TL;DR

Pick the loss with the output layer, the algorithm by data type (GBDT for tables, pretrained nets for unstructured), and the metric by cost of errors — then verify with a baseline and CV.

Last-minute checklist

- ⬜ Framing first: decision, task type, metric tied to business value, **baseline** — before any model.
- ⬜ Train / validation / test: test used **once**; CV reported as mean ± std; stratified, group or time splits to match deployment.
- ⬜ Leakage: "would I know this at prediction time?"; split first; everything inside a `Pipeline`; dedupe across splits.
- ⬜ Two numbers diagnose a model: train ≈ val both bad → underfit; train ≫ val → overfit; both perfect → leakage.
- ⬜ Bias² + variance + noise; regularisation raises bias, lowers variance; bagging ↓ variance, boosting ↓ bias.
- ⬜ $w \leftarrow w - \eta \nabla L$; scale features; Adam(W) lr 1e-3; warmup + cosine; overfit one batch first; initial loss ≈ ln K.
- ⬜ Ridge shrinks, Lasso zeroes; C = 1/strength; logistic boundary $w \cdot x + b = 0$, e^w = odds ratio.
- ⬜ Metrics: precision TP/(TP+FP), recall TP/(TP+FN), F1 harmonic; PR-AUC for rare positives; NDCG for graded lists; MAE vs RMSE + p90 + cohorts.
- ⬜ Threshold from costs: p > C_FP/(C_FP + C_FN); fraud/moderation use two thresholds.
- ⬜ Calibration ≠ ranking: reliability diagram, Brier/log loss, Platt (small) or isotonic (large); recalibrate after downsampling: $p \cdot w/(p \cdot w + 1 - p)$.
- ⬜ Tabular → GBDT with small lr + early stopping; explain with permutation importance or SHAP, not MDI.
- ⬜ Scale for k-NN, SVM, linear, NN, PCA, k-means — never needed for trees.
- ⬜ Attention = $\text{softmax}(QK^T/\sqrt{d_k})V$; $\sqrt{d_k}$ prevents saturation; causal mask for decoders; positional encodings because attention ignores order.
- ⬜ Conv output $\lfloor (n + 2p - k)/s \rfloor + 1$; params $k \cdot k \cdot C_{in} \cdot C_{out} + C_{out}$; start from a pretrained backbone.
- ⬜ Recommenders: funnel (retrieve $10^8 \to 10^3$, rank → 10^2, re-rank → 10–50); two-tower + ANN; time split; popularity baseline; cold start via content + exploration.
- ⬜ RL: Bellman = reward + $\gamma \cdot$next value; Q-learning off-policy, SARSA on-policy; DQN = replay + target net; PPO clipped; RLHF = SFT → RM → PPO + KL.
- ⬜ Design round: 7 steps (3/5/5/5/12/8/5 min); business metric → observable ML objective + guardrail; product knobs outside the model.
- ⬜ Napkin math: QPS = per day ÷ 86,400 (peak ×2–3); vectors N × dims × 4 B; cost = requests × share × calls × price, per completed task.
- ⬜ Production: one feature definition, point-in-time joins, log serving features; batch vs online by need for request-time data; timeout + fallback.
- ⬜ Release: offline gate (per segment) → shadow → canary → ramp; A/B with one primary metric, guardrails, power, whole weeks, no peeking.
- ⬜ Monitoring: health → inputs (PSI > 0.25 investigate, nulls) → predictions → outcomes; alerts start investigations; never auto-promote.

- ☐ Fairness: per-group selection rate, TPR, FPR, PPV, calibration with CIs + proxy test; criteria conflict with unequal base rates — choose by harm.
- ☐ Privacy/security: pseudonymised ≠ anonymous; DP noise $\Delta f/\varepsilon$; adversarial training; label-only APIs, rate limits.
- ☐ Every number next to a baseline; every design has a fallback; every cost in the same unit as the benefit.